

IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

Lecture 6

☒ Introduction to classification

☐ Ensemble Learning

☐ Support Vector Machine

☐ Imbalanced classification

Reading Material:

☐ DM Chapter 4,5

Introduction

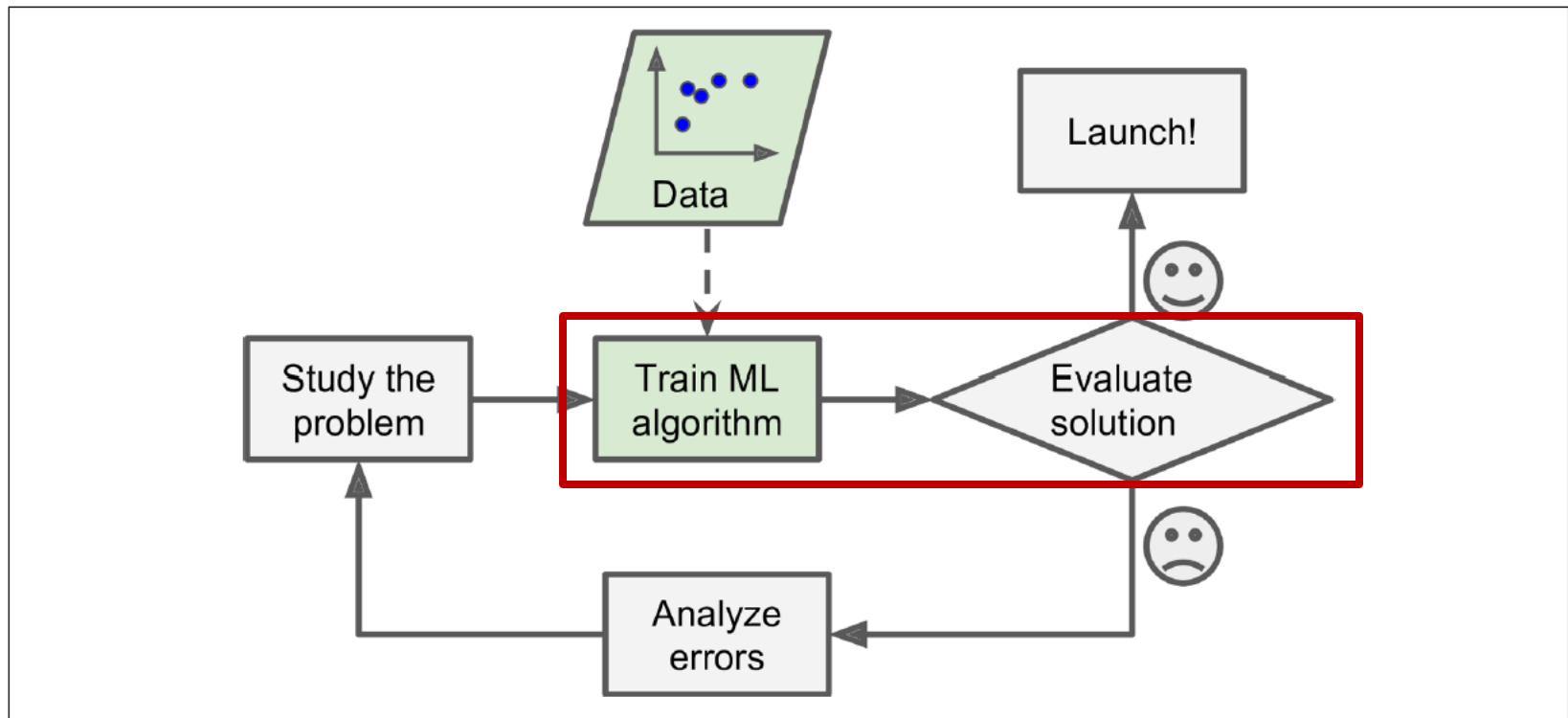


Figure 1-2. Machine Learning approach

Classification: Definition

- ❖ Given a collection of records (training set)
 - ❧ Each record is by characterized by a tuple (x, y) , where x is the attribute set and y is the class label
 - ❖ x : attribute, predictor, independent variable, input
 - ❖ y : class, response, dependent variable, output
- ❖ Task:
 - ❧ Learn a model that maps each attribute set x into one of the predefined **class labels** y

Examples of Classification Task

Task	Attribute set, x	Class label, y
Categorizing email messages	Features extracted from email message header and content	spam or non-spam
Insurance claim	Features extracted from claim and applicant information	Valid claim or not
Credit Card Application	Features extracted from applicants	Approve application or not approve

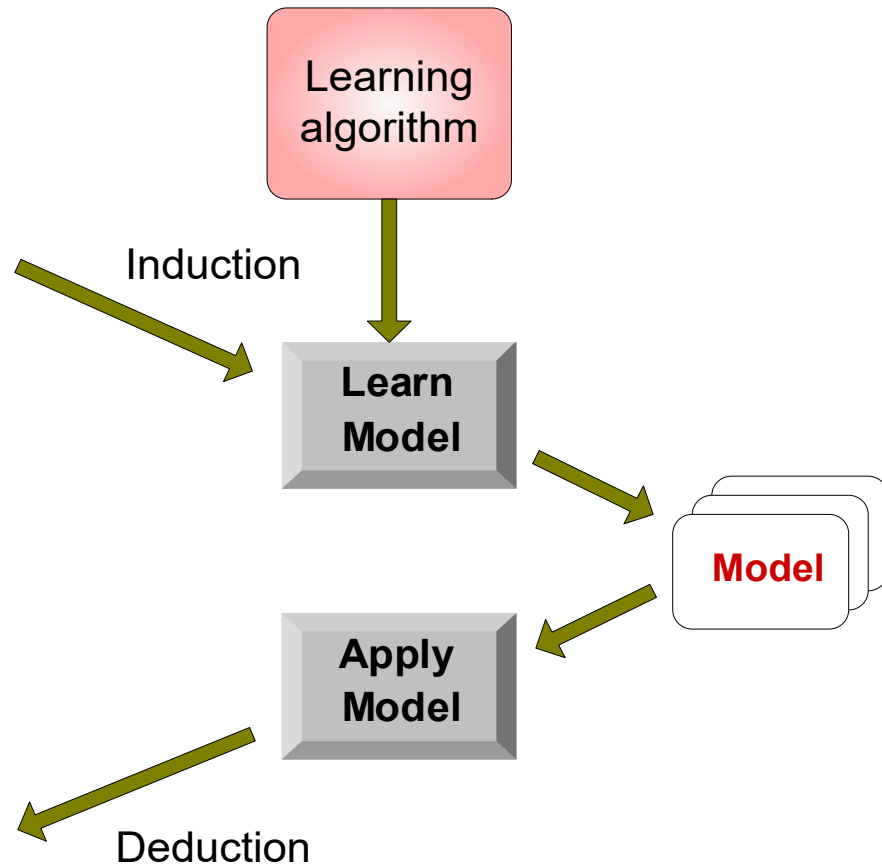
General Approach for Building Classification Model

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



Classification Techniques

❖ Base Classifiers

- ❧ Decision Tree based Methods

- ❧ Rule-based Methods

- ❧ Nearest-neighbor

- ❧ Neural Networks

- ❧ Naïve Bayes and Bayesian Belief Networks

- ❧ Support Vector Machines

❖ Ensemble Classifiers

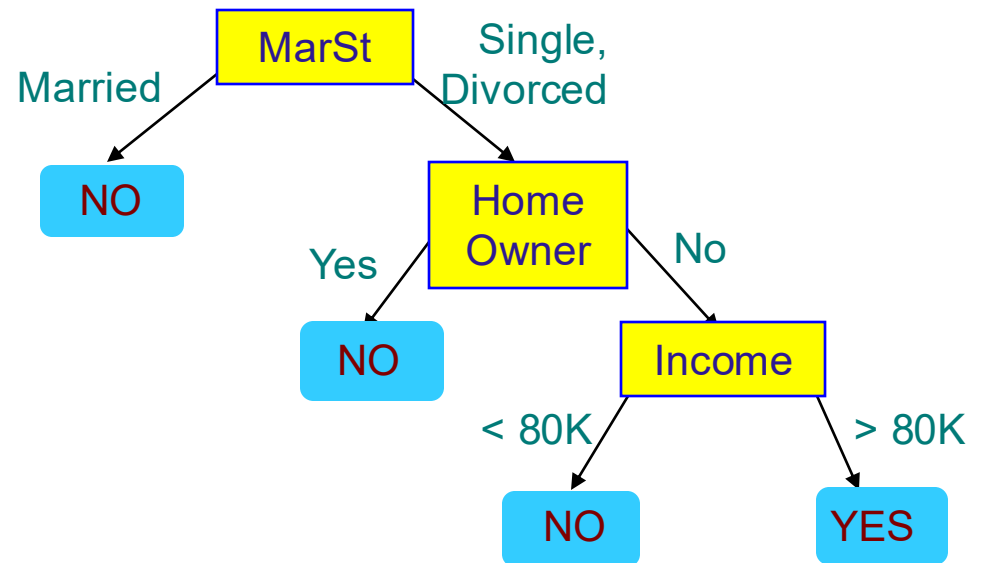
- ❧ Boosting, Bagging, Random Forests

- ❧ Stacked Generalization

Another Example of Decision Tree

categorical
categorical
continuous
class

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

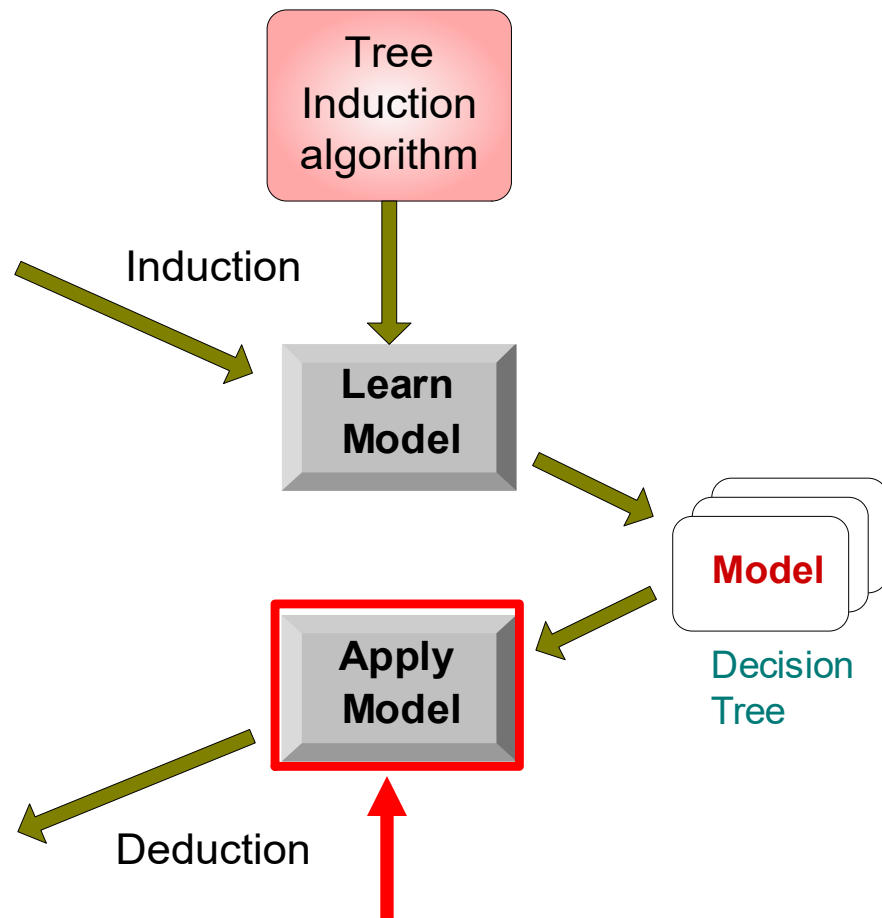
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

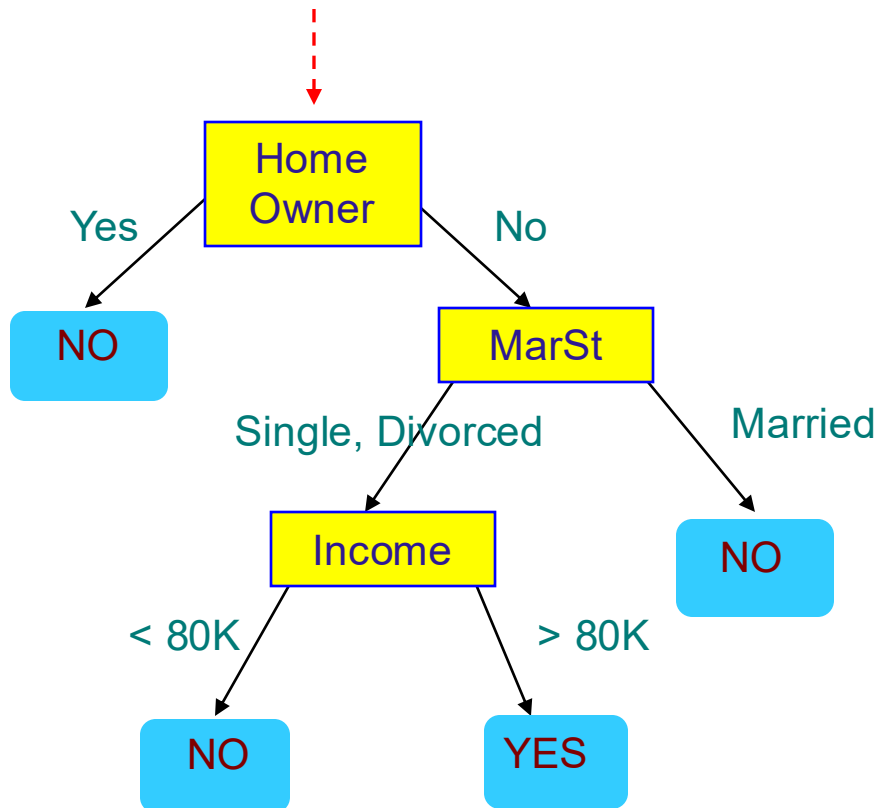
Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



Apply Model to Test Data

Start from the root of tree.



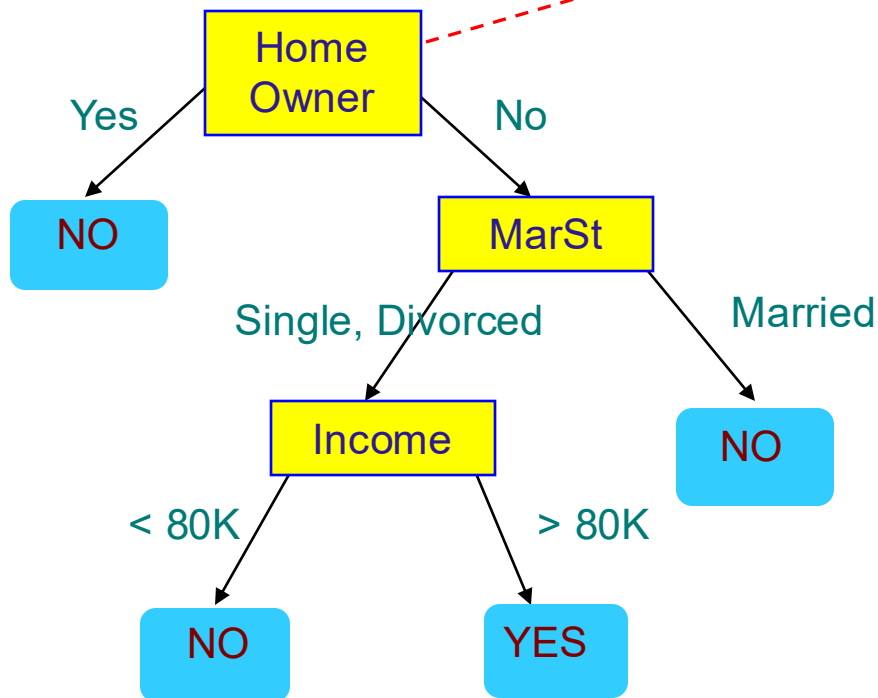
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Apply Model to Test Data

Test Data

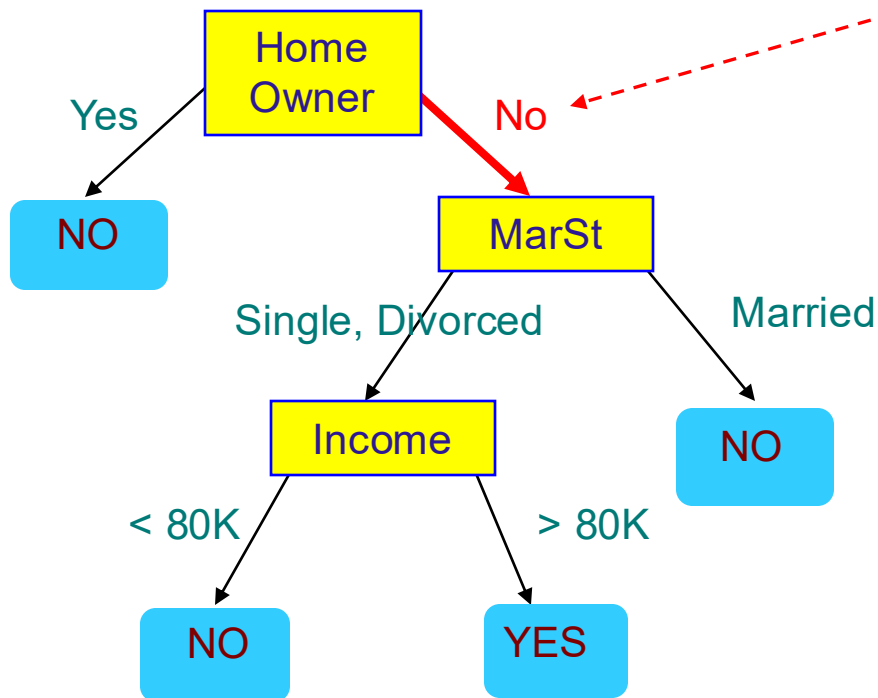
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

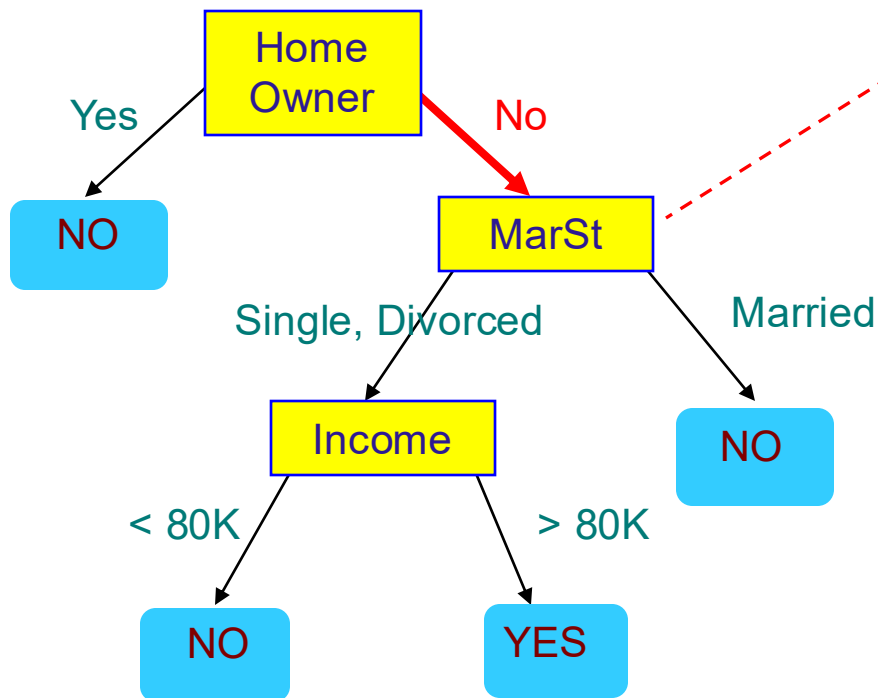
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

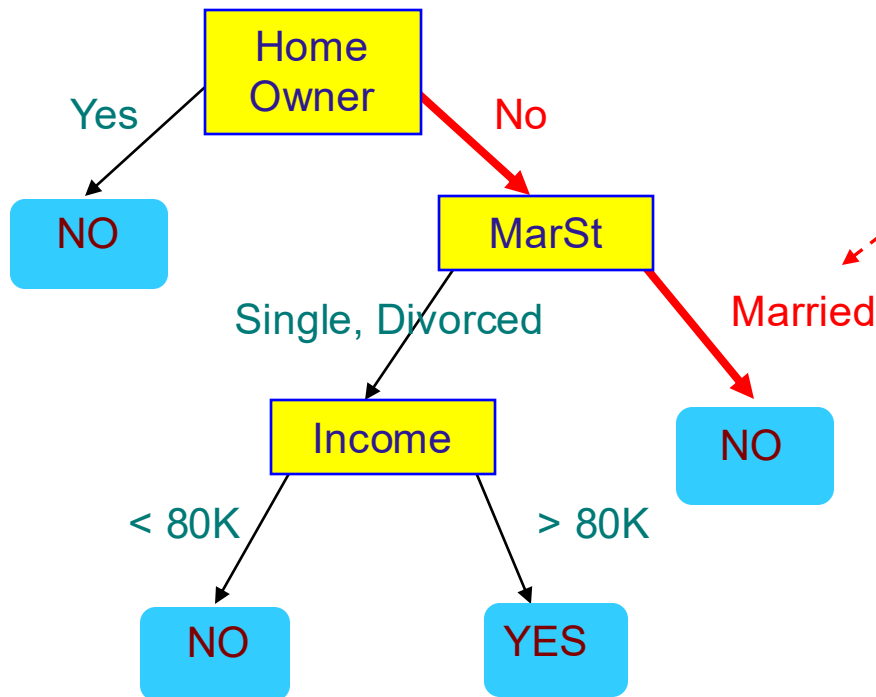
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

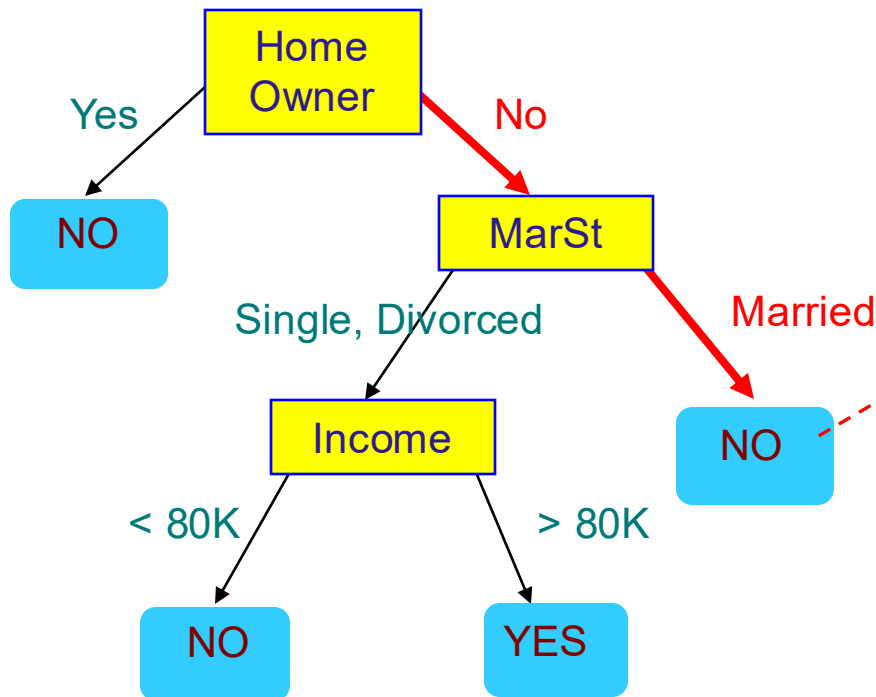
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Assign Defaulted to
"No"

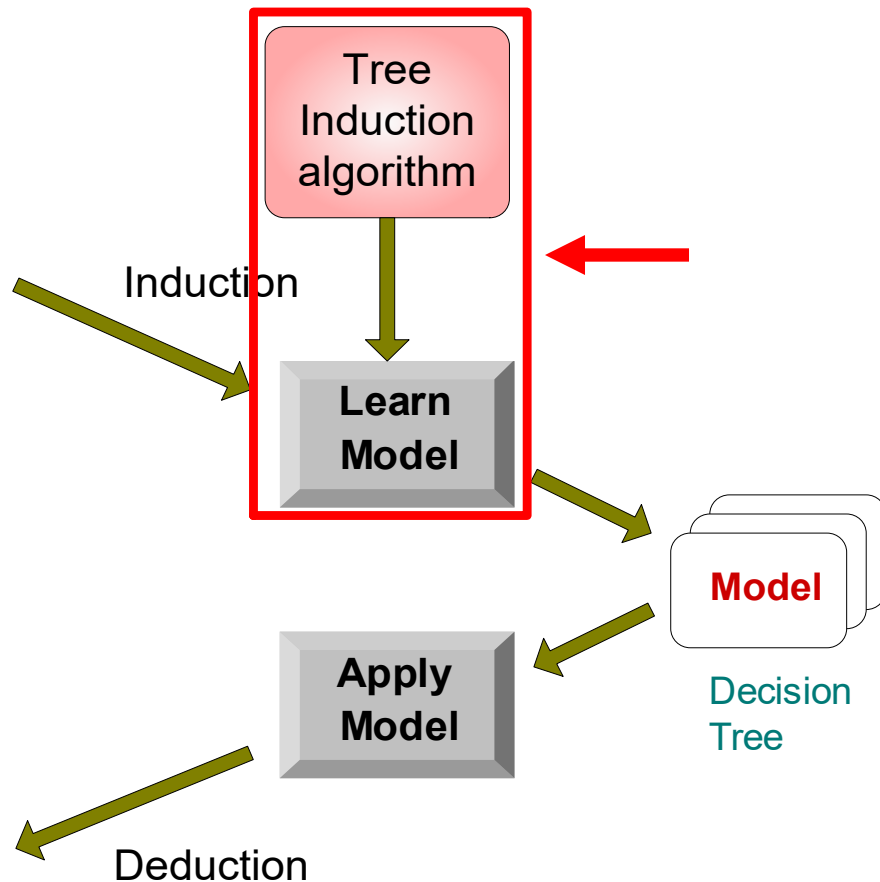
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



Decision Tree Induction

❖ Many Algorithms:

- ❧ Hunt's Algorithm (one of the earliest)

- ❧ CART

- ❧ ID3, C4.5

- ❧ SLIQ, SPRINT

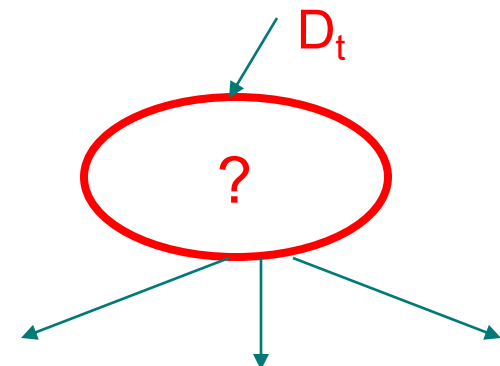
The idea is the same:

Link the features to the right prediction label

Hunt's Algorithm – Tree Construction

- ❖ Let D_t be the set of training records that reach a node t
- ❖ General Procedure:
 - ⌘ If D_t contains records that belong the same class y_t , then t is a leaf node labeled as y_t
 - ⌘ If D_t contains records that belong to more than one class, use an attribute test to split the data into smaller subsets. Recursively apply the procedure to each subset.

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



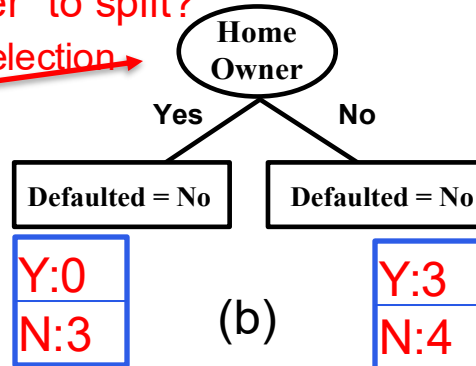
Hunt's Algorithm

Why do we choose "home owner" to split?

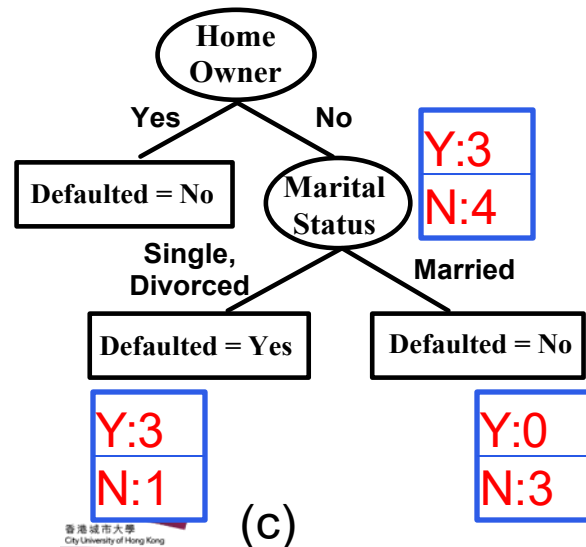
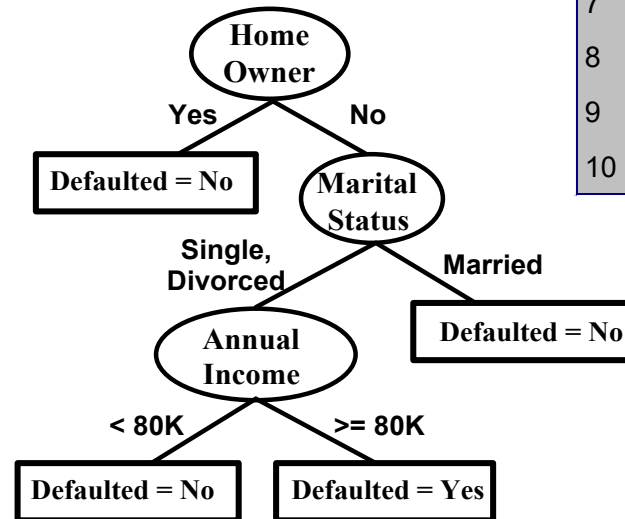
Recall Information Gain in feature selection

Defaulted = No

Y:3
N:7 (a)



ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Decision Tree Based Classification

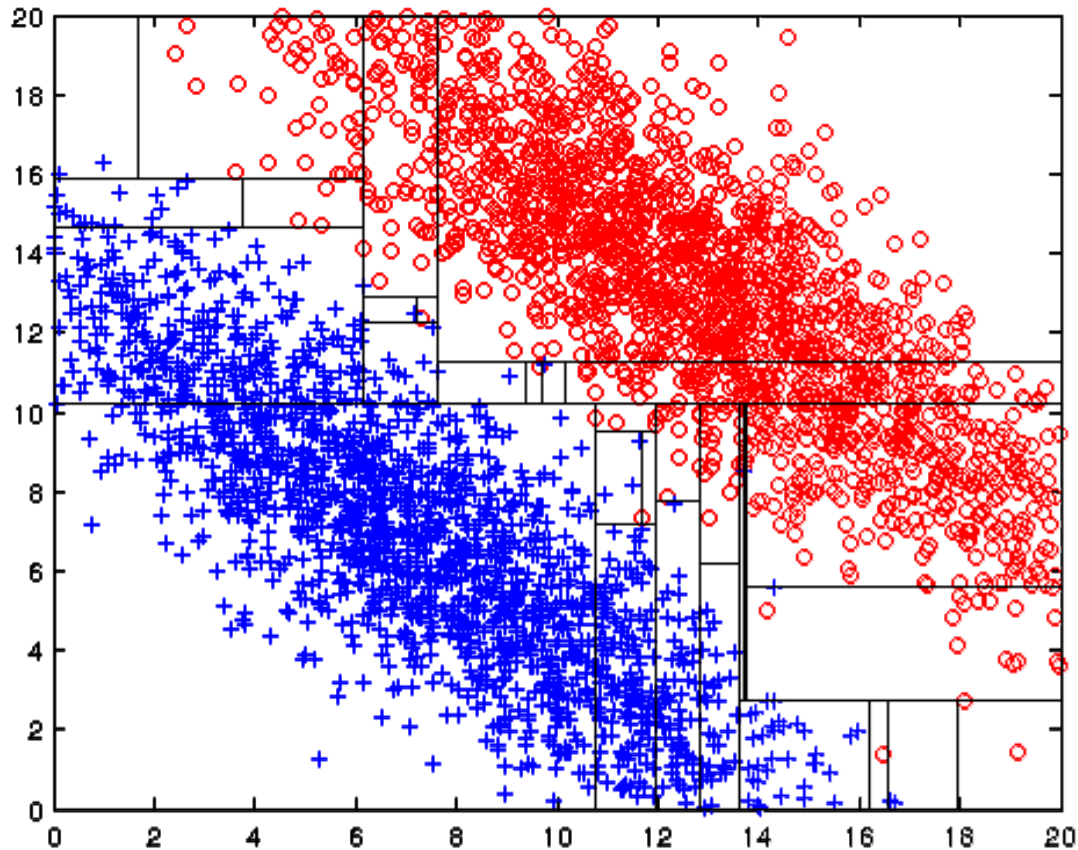
❖ Advantages:

- ❧ Inexpensive to construct
- ❧ Extremely fast at classifying unknown records
- ❧ Easy to interpret for small-sized trees
- ❧ Robust to noise (especially when methods to avoid overfitting are employed)
- ❧ Can easily handle redundant or irrelevant attributes (unless the attributes are interacting)

❖ Disadvantages:

- ❧ Space of possible decision trees is exponentially large. Greedy approaches are often unable to find the best tree.
- ❧ Does not take into account interactions between attributes
- ❧ Each decision boundary involves only a single attribute

Limitations of single attribute-based decision boundaries



Both positive (+) and negative (o) classes generated from skewed Gaussians with centers at (8,8) and (12,12) respectively.

Example Data Set

Two class problem:

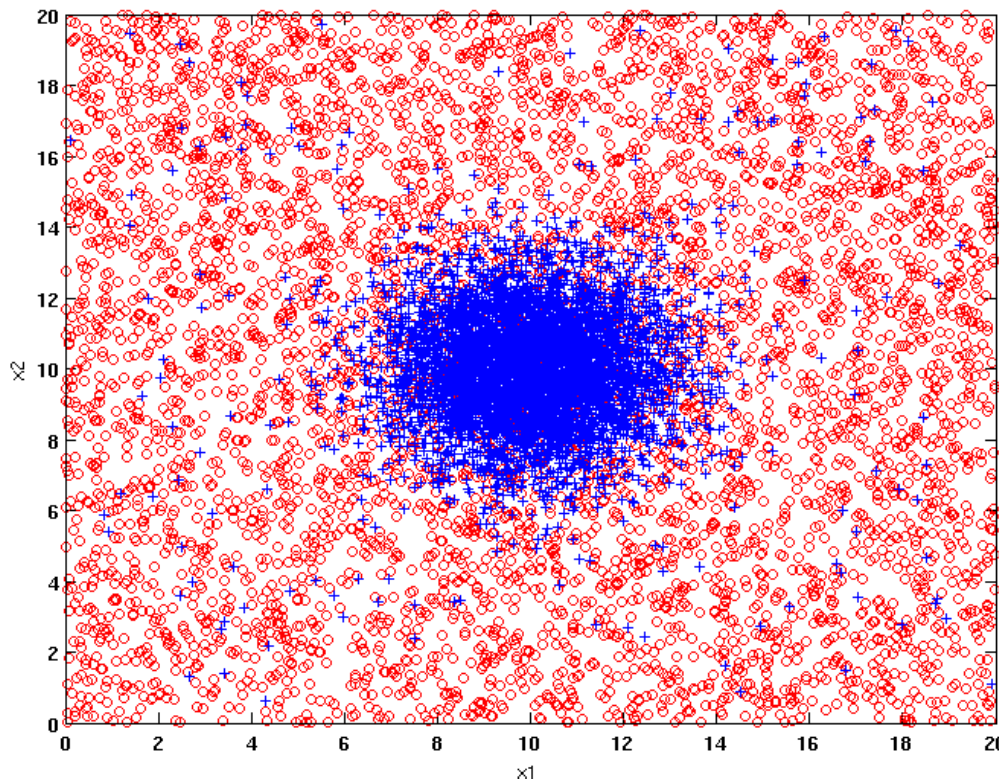
+ : 5200 instances

- 5000 instances generated from a Gaussian centered at (10,10)
- 200 noisy instances added

o : 5200 instances

- Generated from a uniform distribution

10 % of the data used for training and 90% of the data used for testing



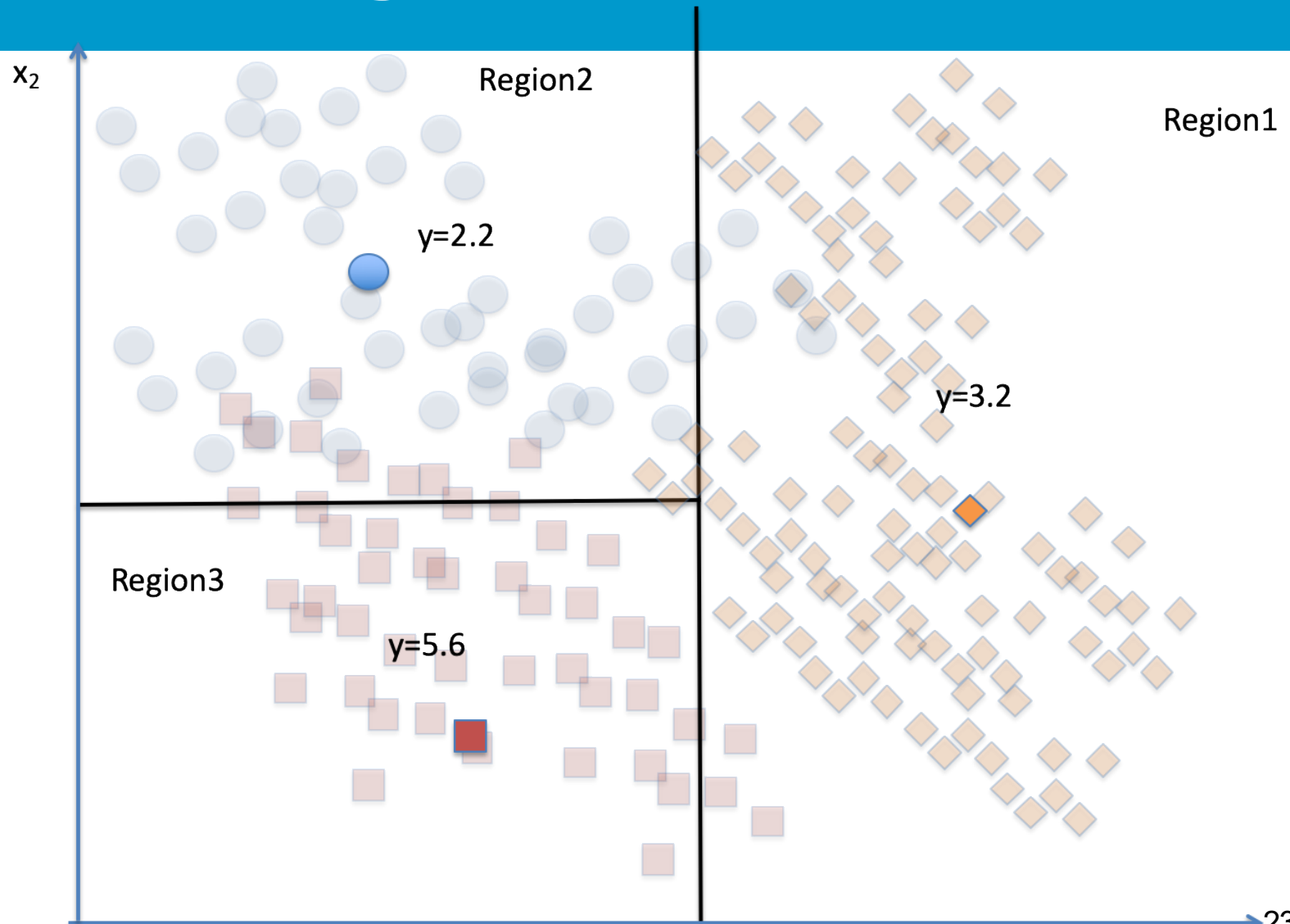
Regression Tree

Build a regression tree:

Divide the predictor space into J distinct not overlapping regions $R_1, R_2, R_3, \dots, R_J$

We make the same prediction for all observations in the same region; use the mean of responses for all training observations that are in the region

Regression Tree



IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

Lecture 6

☐ Introduction to classification

☒ **Ensemble Learning**

☐ Support Vector Machine

☐ Imbalanced classification

Reading Material:

☐ DM Chapter 4,5

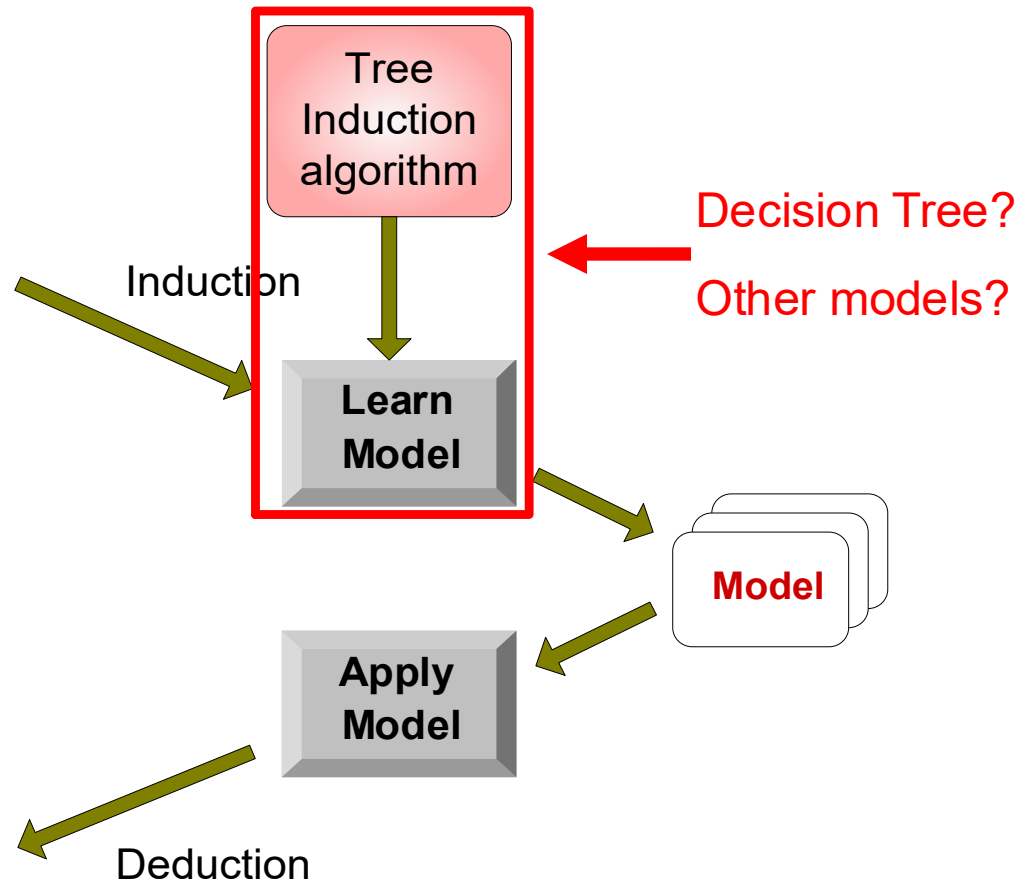
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



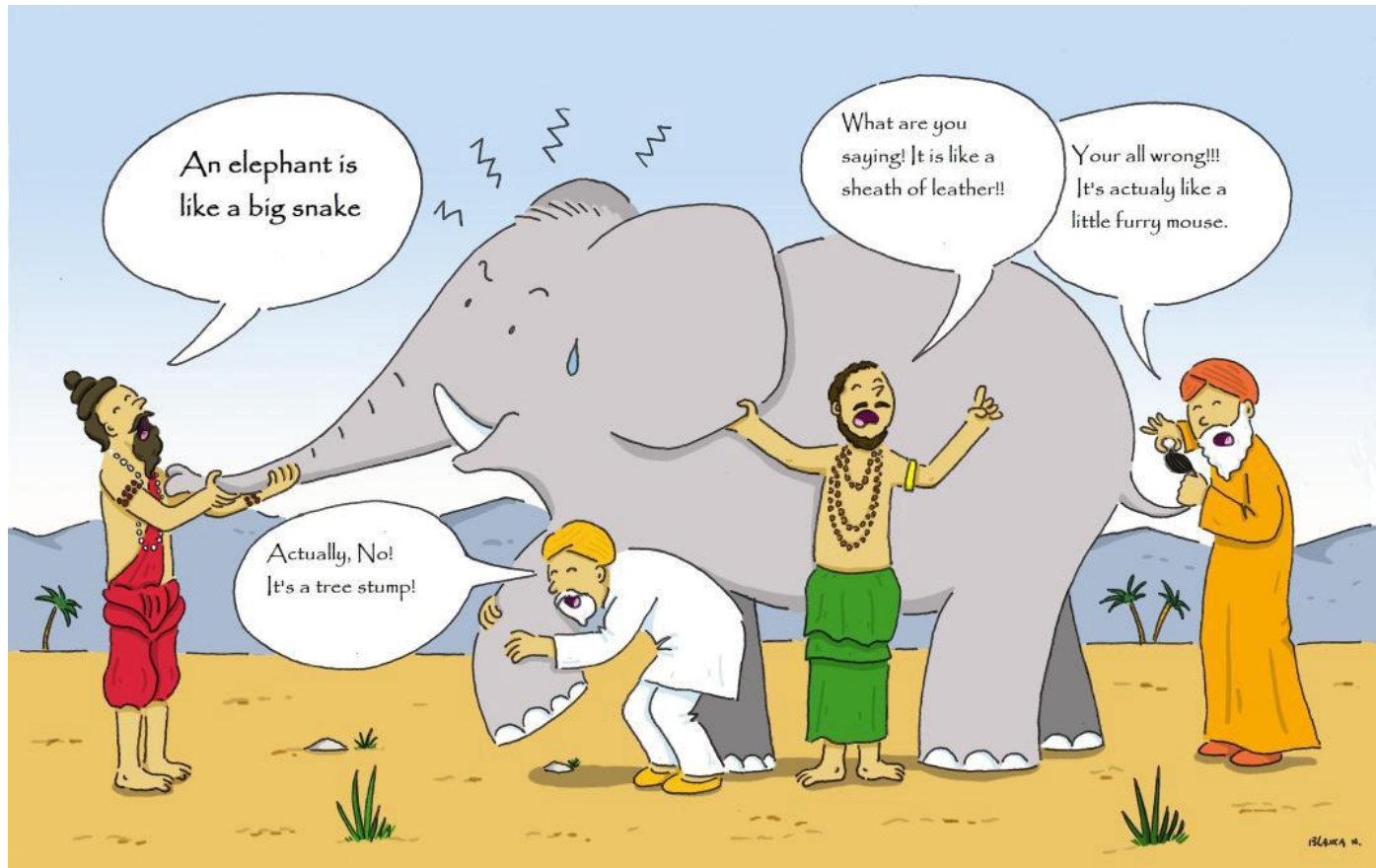
Ensemble Methods

- ❖ Construct a set of classifiers from the training data
- ❖ Predict class label of test records by combining the predictions made by multiple classifiers

Ensemble Methods

After everybody takes a guess...

I guess.....

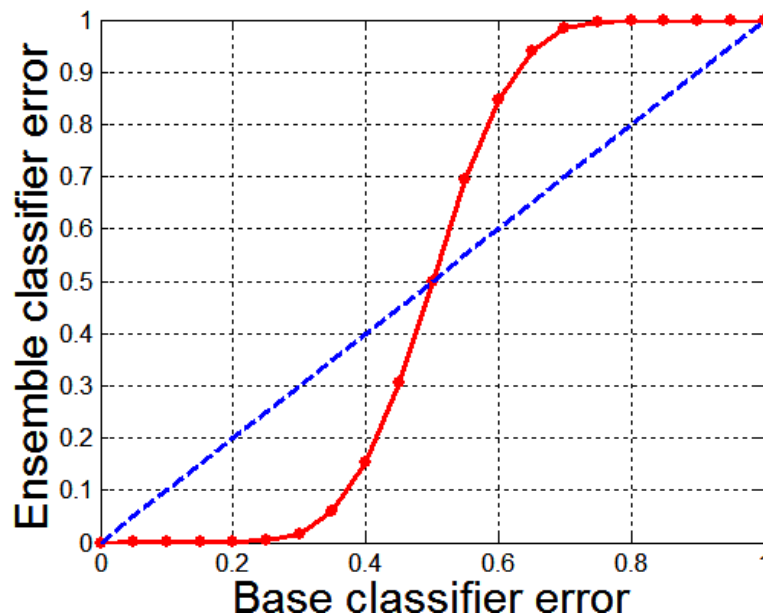


BLIND MEN & ELEPHANT PARABLE. Source

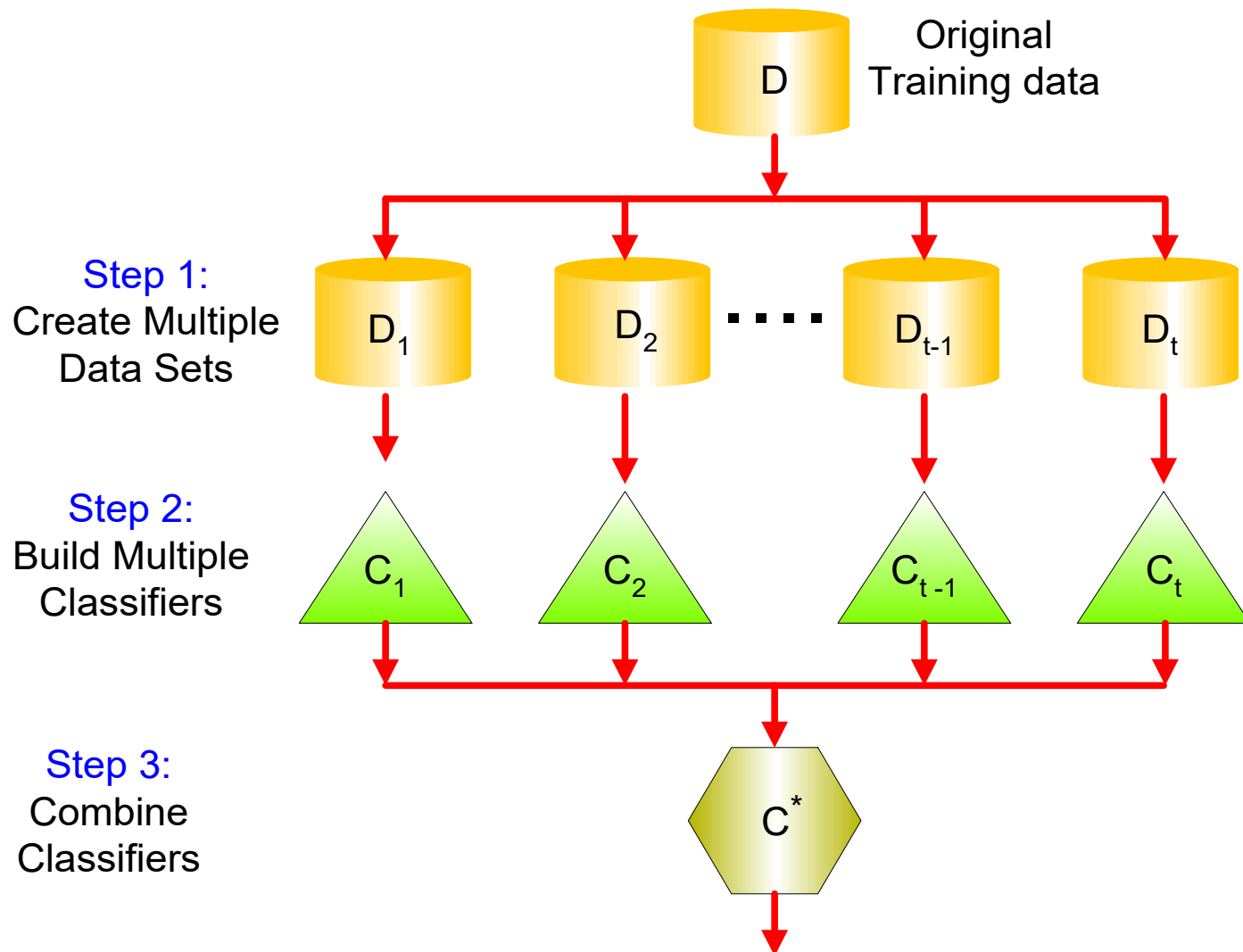
Why Ensemble Methods work?

- ❖ Suppose there are 25 base classifiers, we guess according to the majority outputs
 - ⌘ if most guess +, we predict +
 - ⌘ Each classifier has error rate, $\varepsilon = 0.35$
 - ⌘ Assume errors made by classifiers are uncorrelated
 - ⌘ Probability that the ensemble classifier makes a wrong prediction:

$$P(X \geq 13) = \sum_{i=13}^{25} \binom{25}{i} \varepsilon^i (1 - \varepsilon)^{25-i} = 0.06$$



General Approach



Types of Ensemble Methods

- ❖ Bayesian ensemble
 - ❧ Example: Mixture of Gaussian
- ❖ Manipulate data distribution
 - ❧ Example: Resampling method
- ❖ Manipulate input features
 - ❧ Example: Feature subset selection
- ❖ Manipulate class labels
 - ❧ Example: error-correcting output coding
- ❖ Introduce randomness into learning algorithm
 - ❧ Example: Random forests

Bagging

❖ Sampling with replacement

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

❖ Build classifier on each bootstrap sample

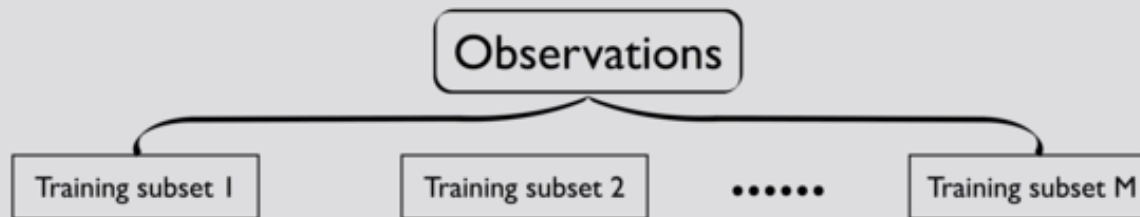
Bagging Algorithm

Algorithm 5.6 Bagging Algorithm

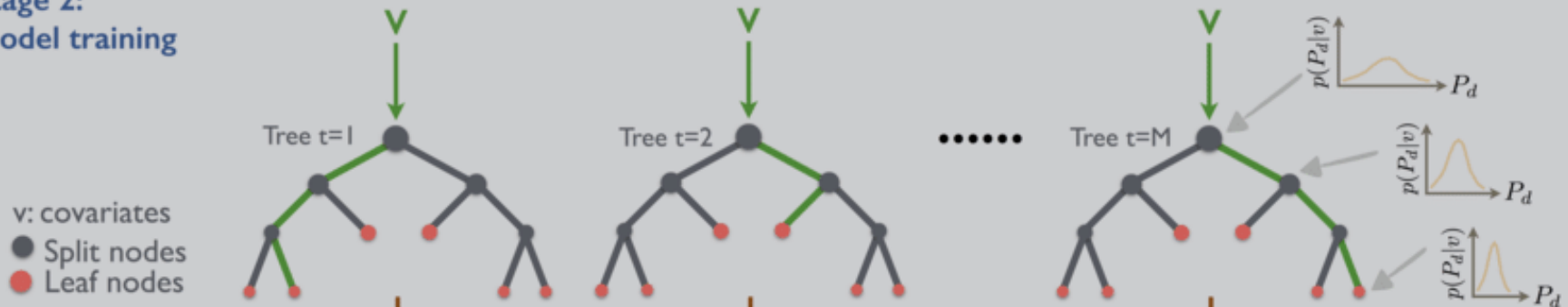
- 1: Let k be the number of bootstrap samples.
 - 2: for $i = 1$ to k do
 - 3: Create a bootstrap sample of size n , D_i .
 - 4: Train a base classifier C_i on the bootstrap sample D_i .
 - 5: end for
 - 6: $C^*(x) = \arg \max_y \sum_i \delta(C_i(x) = y)$, $\{\delta(\cdot) = 1$ if its argument is true, and 0 otherwise. $\}$
-

Random Forest

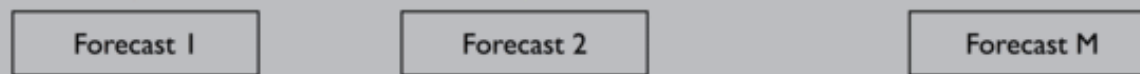
Stage 1:
Bootstrap sampling



Stage 2:
Model training



Stage 3:
Model forecasting



Stage 4:
Result aggregating



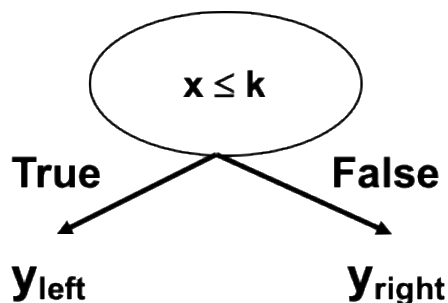
Bagging Example

- Consider 1-dimensional data set:

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

- Classifier is a decision stump
 - Decision rule: $x \leq k$ versus $x > k$
 - Split point k is chosen based on entropy



Bagging Example

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.5	0.9	1	1	1
y	1	1	1	-1	-1	-1	1	1	1	1

$x \leq 0.7 \rightarrow y = 1$

$x > 0.7 \rightarrow y = 1$

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.3 \rightarrow y = 1$

$x > 0.3 \rightarrow y = -1$

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Example

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

$x \leq 0.05 \rightarrow y = 1$

$x > 0.05 \rightarrow y = 1$

Bagging Example

❖ Summary of Training sets:

$X=0.5$

Round	Split Point	Left Class	Right Class	Prediction
1	0.35	1	-1	-1
2	0.7	1	1	1
3	0.35	1	-1	-1
4	0.3	1	-1	-1
5	0.35	1	-1	-1
6	0.75	-1	1	-1
7	0.75	-1	1	-1
8	0.75	-1	1	-1
9	0.75	-1	1	-1
10	0.05	1	1	1

Bagging Example

- ❖ Assume test set is the same as the original data
- ❖ Use majority vote to determine class of ensemble classifier

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7
1	1	1	1	-1	-1	-1	-1
2	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1
7	-1	-1	-1	-1	-1	-1	-1
8	-1	-1	-1	-1	-1	-1	-1
9	-1	-1	-1	-1	-1	-1	-1
10	1	1	1	1	1	1	1
Predicted Class	2	2	2	-6	-6	-6	-6
Sign	1	1	1	-1	-1	-1	-1

Round	Split Point	Left Class	Right Class
1	0.35	1	-1
2	0.7	1	1
3	0.35	1	-1
4	0.3	1	-1
5	0.35	1	-1
6	0.75	-1	1
7	0.75	-1	1
8	0.75	-1	1
9	0.75	-1	1
10	0.05	1	1

Bagging Example

- ❖ Quiz: fill the table for column $x=0.9$. What is the predicted result? Can this model well fit the original data?

Round	$x=0.1$	$x=0.2$	$x=0.3$	$x=0.4$	$x=0.5$	$x=0.6$	$x=0.7$
1	1	1	1	-1	-1	-1	-1
2	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1
7	-1	-1	-1	-1	-1	-1	-1
8	-1	-1	-1	-1	-1	-1	-1
9	-1	-1	-1	-1	-1	-1	-1
10	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6
Sign	1	1	1	-1	-1	-1	-1

Round	Split Point	Left Class	Right Class
1	0.35	1	-1
2	0.7	1	1
3	0.35	1	-1
4	0.3	1	-1
5	0.35	1	-1
6	0.75	-1	1
7	0.75	-1	1
8	0.75	-1	1
9	0.75	-1	1
10	0.05	1	1

Predicted
Class

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

Boosting

- ❖ An iterative procedure to adaptively change distribution of training data by focusing more on previously misclassified records
 - ∞ Initially, all N records are assigned equal weights
 - ∞ Unlike bagging, weights may change at the end of each boosting round

Boosting

- ❖ Records that are wrongly classified will have their weights increased
- ❖ Records that are classified correctly will have their weights decreased

Original Data	1	2	3	4	5	6	7	8	9	10
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

- Example 4 is hard to classify
- Its weight is increased; therefore it is more likely to be chosen again in subsequent rounds

AdaBoost Algorithm

❖ Base classifiers: $C_1, C_2, \dots, C_T$

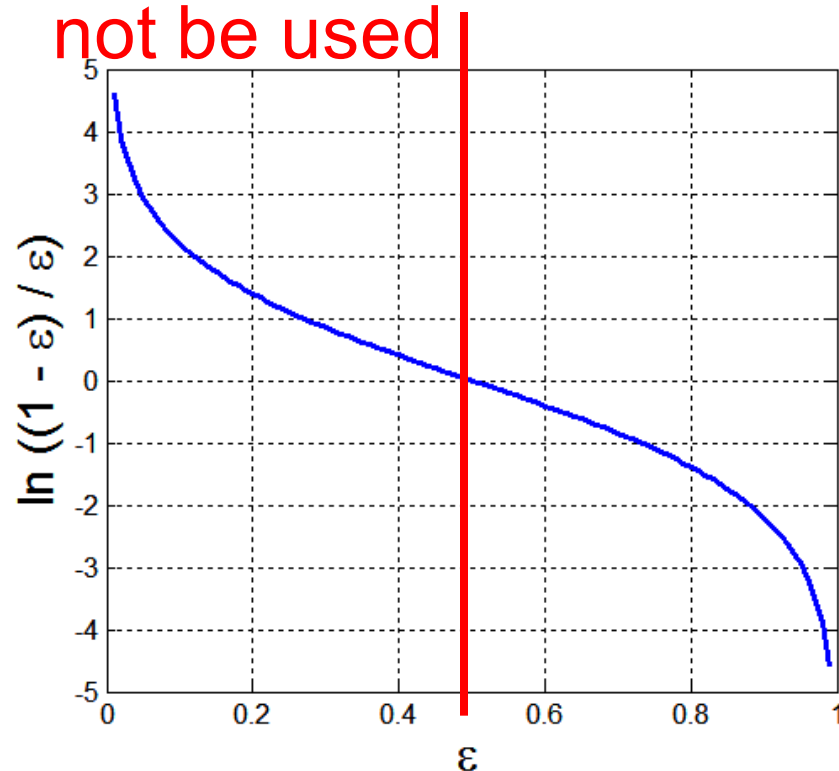
❖ Error rate:

$$\varepsilon_i = \frac{1}{N} \sum_{j=1}^N w_j \delta(C_i(x_j) \neq y_j)$$

❖ Importance of a classifier:

$$\alpha_i = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_i}{\varepsilon_i} \right)$$

Base error rate > 0.5 will not be used



AdaBoost Algorithm

❖ Weight update:

$$w_i^{(j+1)} = \frac{w_i^{(j)}}{Z_j} \begin{cases} \exp^{-\alpha_j} & \text{if } C_j(x_i) = y_i \\ \exp^{\alpha_j} & \text{if } C_j(x_i) \neq y_i \end{cases}$$

where Z_j is the normalization factor

- ❖ If any intermediate rounds produce error rate higher than 50%, the weights are reverted back to $1/n$ and the resampling procedure is repeated

 Classification: $C^*(x) = \arg \max_y \sum_{j=1}^T \alpha_j \delta(C_j(x) \neq y)$

AdaBoost Algorithm

Algorithm 5.7 AdaBoost Algorithm

- 1: $w = \{w_j = 1/n \mid j = 1, 2, \dots, n\}$. {Initialize the weights for all n instances.}
 - 2: Let k be the number of boosting rounds.
 - 3: for $i = 1$ to k do
 - 4: Create training set D_i by sampling (with replacement) from D according to w .
 - 5: Train a base classifier C_i on D_i .
 - 6: Apply C_i to all instances in the original training set, D .
 - 7: $\epsilon_i = \frac{1}{n} [\sum_j w_j \delta(C_i(x_j) \neq y_j)]$ {Calculate the weighted error}
 - 8: if $\epsilon_i > 0.5$ then
 - 9: $w = \{w_j = 1/n \mid j = 1, 2, \dots, n\}$. {Reset the weights for all n instances.}
 - 10: Go back to Step 4.
 - 11: end if
 - 12: $\alpha_i = \frac{1}{2} \ln \frac{1-\epsilon_i}{\epsilon_i}$.
 - 13: Update the weight of each instance according to equation (5.88).
 - 14: end for
 - 15: $C^*(x) = \arg \max_y \sum_{j=1}^T \alpha_j \delta(C_j(x) = y)$.
-

AdaBoost Example

- ❖ Consider 1-dimensional data set:

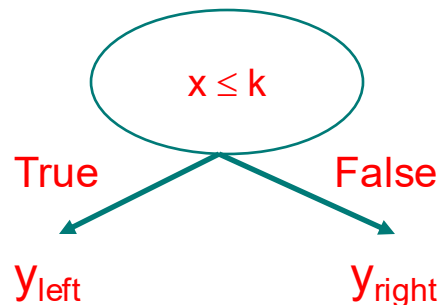
Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

- ❖ Classifier is a decision stump

⌘ Decision rule: $x \leq k$ versus $x > k$

⌘ Split point k is chosen based on entropy



AdaBoost Example

❖ Training sets for the first 3 boosting rounds:

Boosting Round 1:

x	0.1	0.4	0.5	0.6	0.6	0.7	0.7	0.7	0.8	1
y	1	-1	-1	-1	-1	-1	-1	-1	1	1

Boosting Round 2:

x	0.1	0.1	0.2	0.2	0.2	0.2	0.3	0.3	0.3	0.3
y	1	1	1	1	1	1	1	1	1	1

Boosting Round 3:

x	0.2	0.2	0.4	0.4	0.4	0.4	0.5	0.6	0.6	0.7
y	1	1	-1	-1	-1	-1	-1	-1	-1	-1

❖ Summary:

Round	Split Point	Left Class	Right Class	alpha
1	0.75	-1	1	1.738
2	0.05	1	1	2.7784
3	0.3	1	-1	4.1195

AdaBoost Example

❖ Weights

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
2	0.311	0.311	0.311	0.01	0.01	0.01	0.01	0.01	0.01	0.01
3	0.029	0.029	0.029	0.228	0.228	0.228	0.228	0.009	0.009	0.009

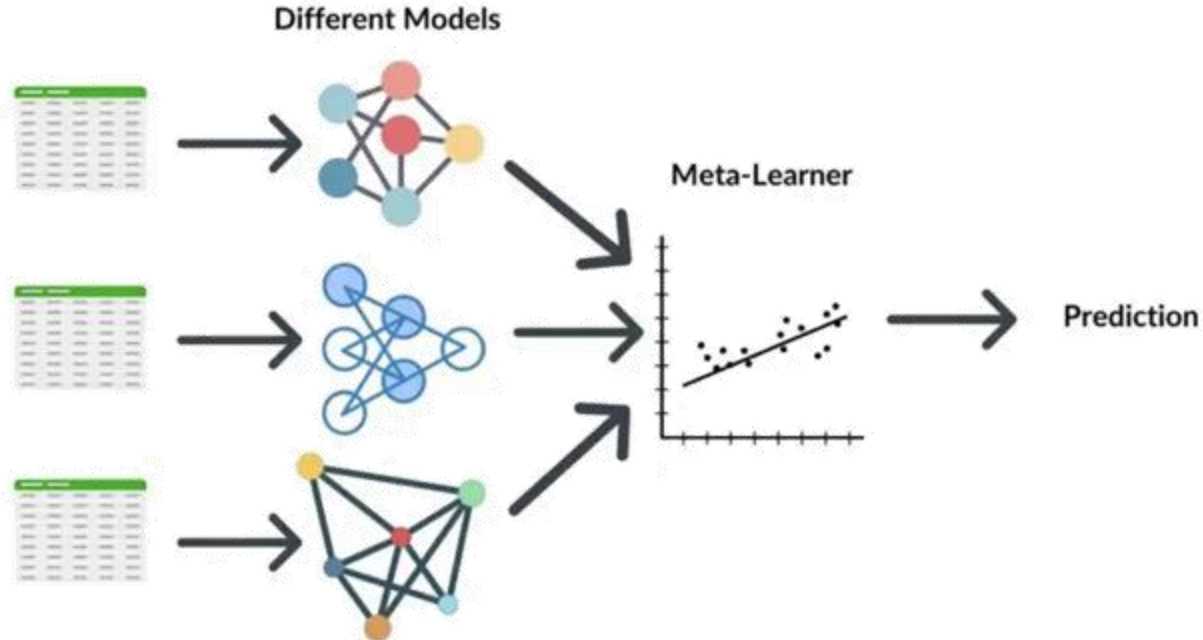
❖ Classification

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	-1	-1	-1	-1	-1	-1	-1	1	1	1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
Sum	5.16	5.16	5.16	-3.08	-3.08	-3.08	-3.08	0.397	0.397	0.397
Predicted Class	1	1	1	-1	-1	-1	-1	1	1	1

Round	Split Point	Left Class	Right Class	alpha
1	0.75	-1	1	1.738
2	0.05	1	1	2.7784
3	0.3	1	-1	4.1195

Stacking/ Stacked Generalization

- ❖ Combine predictions from
 - ∞ Level-1 (**Base Model**): multiple base models
 - ∞ Level-2 (**Meta-model**): to learn how to best integrate level-1 models' output



Stacking vs. Bagging and Boosting

❖ Advantages

- ∞ Exploits heterogeneity among models
- ∞ Learn optimal combination weights automatically

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

Bagging:
Parallel then average

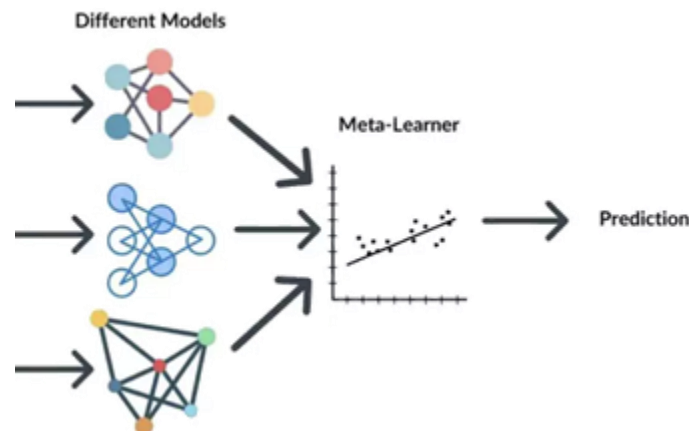
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

Weight (R1)

Weight (R2)

Boosting:
Sequential updated focus

1	2	3	4	5	6	7	8	9	10
7	8	10	8	2	5	10	10	5	9
1	4	9	1	2	3	2	7	3	2
1	8	5	10	5	5	9	6	3	7



Stacking:
Two-level structure

StackingClassifier

Algorithm 1 StackingClassifier (Stacked Generalization for Classification)

Training dataset $D = \{(x_i, y_i)\}_{i=1}^n$;

Base classifiers $\{\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_K\}$;

Meta-classifier $\mathcal{M}$;

Number of folds F for cross-validation. Final stacked classifier $H(x)$.

Step 1: Generate Level-1 Meta-Features

Split D into F disjoint folds $\{D_1, D_2, \dots, D_F\}$

for each base classifier $\mathcal{L}_k$ **do**

for each fold $f = 1, \dots, F$ **do** Train $\mathcal{L}_k^{(-f)}$ on $D \setminus D_f$ Predict on the held-out fold D_f to obtain out-of-fold predictions:

$z_{k,i} = \mathcal{L}_k^{(-f)}.\text{predict_proba}(x_i)$ for all $(x_i, y_i) \in D_f$ Concatenate predictions to form column $Z_k = [z_{k,1}, \dots, z_{k,n}]^T$ Construct meta-level dataset $Z = [Z_1, Z_2, \dots, Z_K] \in \mathbb{R}^{n \times K \times C}$

Step 2: Train Meta-Classifier

Train the meta-classifier $\hat{\mathcal{M}} = \mathcal{M}(Z, y)$ on (Z, y)

Step 3: Retrain Base Classifiers on Full Data

for each base classifier $\mathcal{L}_k$ **do** Retrain $\hat{\mathcal{L}}_k = \mathcal{L}_k.\text{fit}(D)$

Step 4: Make Final Predictions

for a new instance x^* **do** Obtain base-level predictions:

$z_k^* = \hat{\mathcal{L}}_k.\text{predict_proba}(x^*)$, for $k = 1, \dots, K$ Concatenate $z^* = [z_1^*, \dots, z_K^*]^T$ Compute final prediction:

$H(x^*) = \hat{\mathcal{M}}.\text{predict}(z^*)$

Return: Final stacked classifier $H(x)$.

StackingClassifier Example

Sample	x_1	x_2	y
1	0	1	0
2	1	0	0
3	1	1	1
4	2	1	1
5	2	0	1

Base Model1: Logistic Regression

Base Model2: Decision Tree

Meta Model: Logistic Regression

Fold = 2

Fold1 = {1, 2, 3}

Fold2 = {4, 5}

Step1: Training on Base Models

	Training Data	Testing Data on $P(y = 1)$
Base 1 LR	$\{1,2,3\} \rightarrow \{4,5\}$	(4):0.75, (5):0.70
	$\{4,5\} \rightarrow \{1,2,3\}$	(1):0.30, (2):0.35, (3):0.65
Base 2 Tree	$\{1,2,3\} \rightarrow \{4,5\}$	(4):0.95, (5):0.80
	$\{4,5\} \rightarrow \{1,2,3\}$	(1):0.10, (2):0.15, (3):0.85

StackingClassifier Example

Step2: Training on Meta-level Model

Sample	Base 1: LR	Base 2: Tree	y (True value)
1	0.30	0.10	0
2	0.35	0.15	0
3	0.65	0.85	1
4	0.75	0.95	1
5	0.70	0.80	1

$$\hat{y} = f(LR_{pred}, Tree_{pred})$$

$$\text{logit}(y)$$

$$= -4 + 3.0 \times LR + 2.0 \times Tree$$

StackingClassifier Example

Inference on Testing data set

Testing	x_1	x_2	y
A	0	0	?
B	1	1	?
C	2	0	?

	A	B	C
LR	0.25	0.70	0.60
Tree	0.10	0.85	0.80

$$\text{logit}_A = -4 + 3.0(0.25) + 2.0(0.10) = -3.05$$

$$\text{logit}_B = -4 + 3.0(0.70) + 2.0(0.85) = 0.8$$

$$\text{logit}_C = -4 + 3.0(0.60) + 2.0(0.80) = 0.4$$

Sigmoid to get prob

Testing	Meta-logit	$P(y = 1)$	y (True value)
A	-0.35	0.045	0
B	0.8	0.69	1
C	0.4	0.60	1

IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

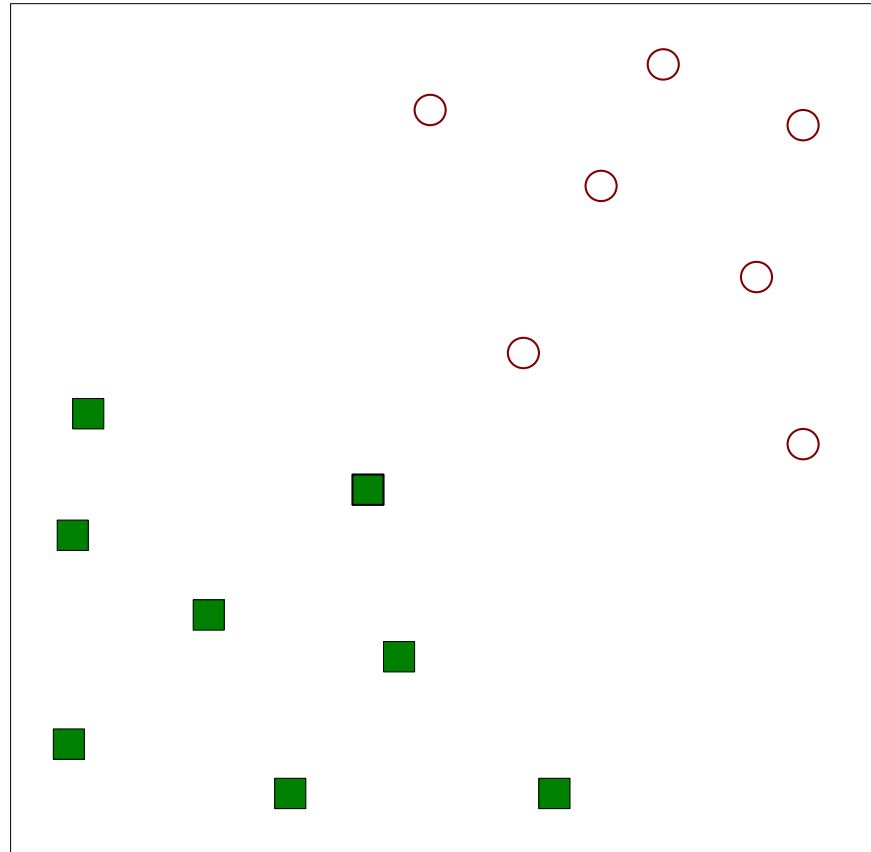
Lecture 6

- ☐ Introduction to classification
- ☐ Ensemble Learning
- ☒ **Support Vector Machine**
- ☐ Imbalanced classification

Reading Material:

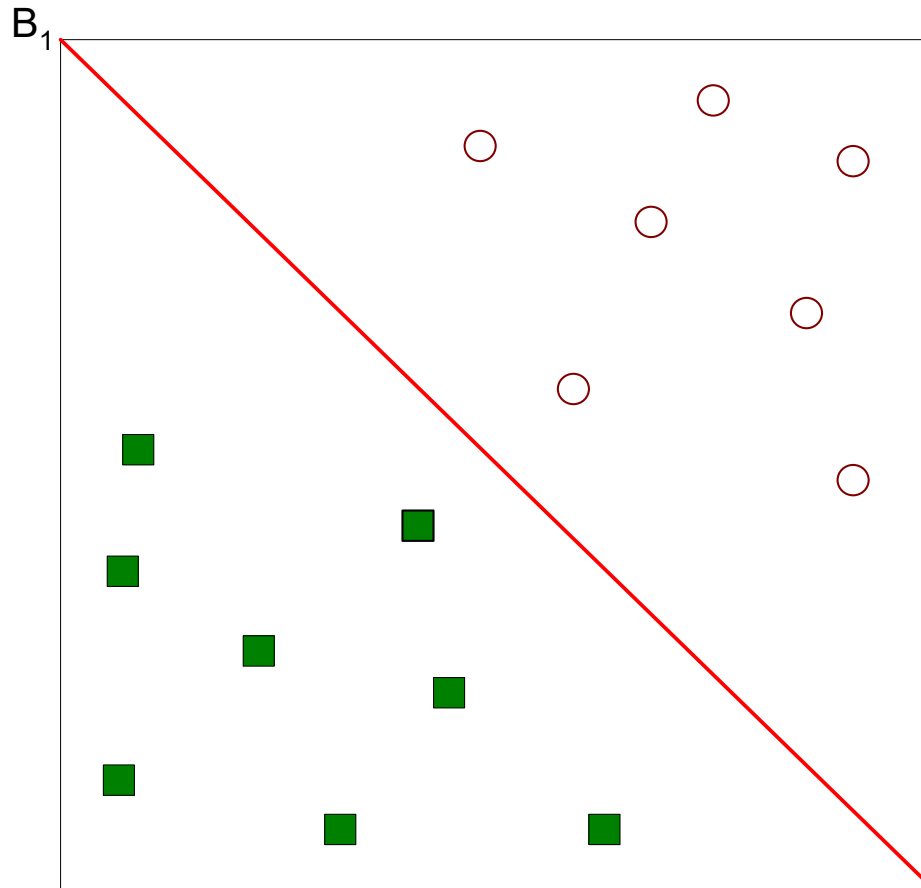
- ☐ DM Chapter 4,5

Support Vector Machines



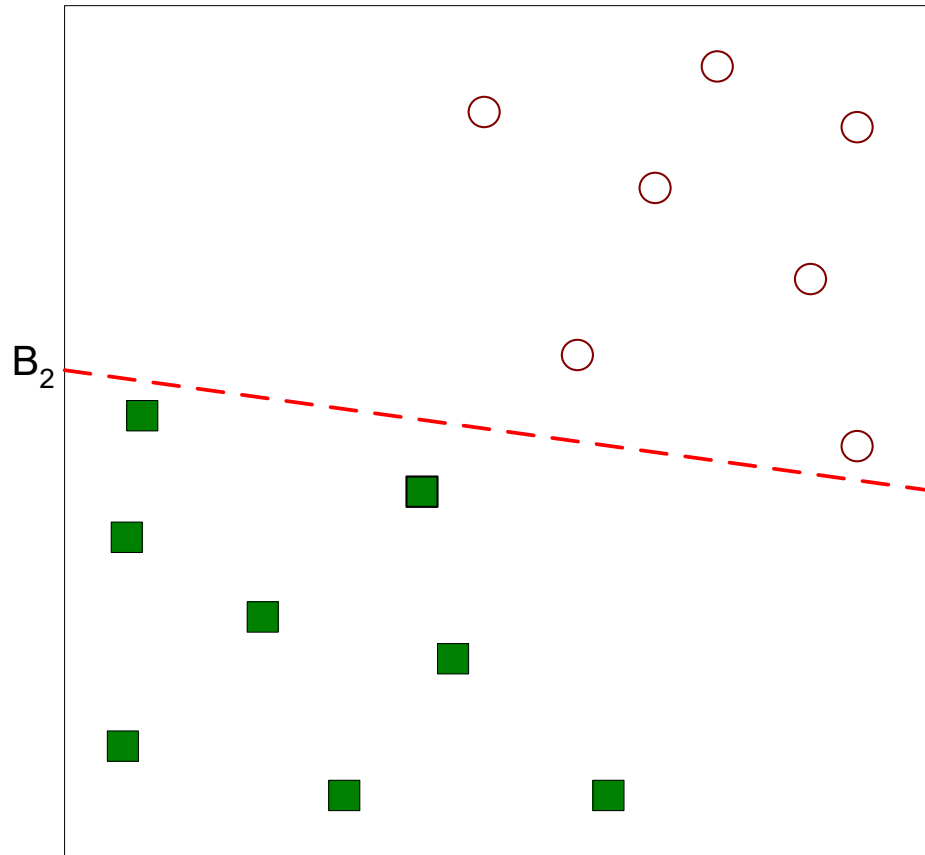
- ❖ Find a linear hyperplane (decision boundary) that will separate the data

Support Vector Machines



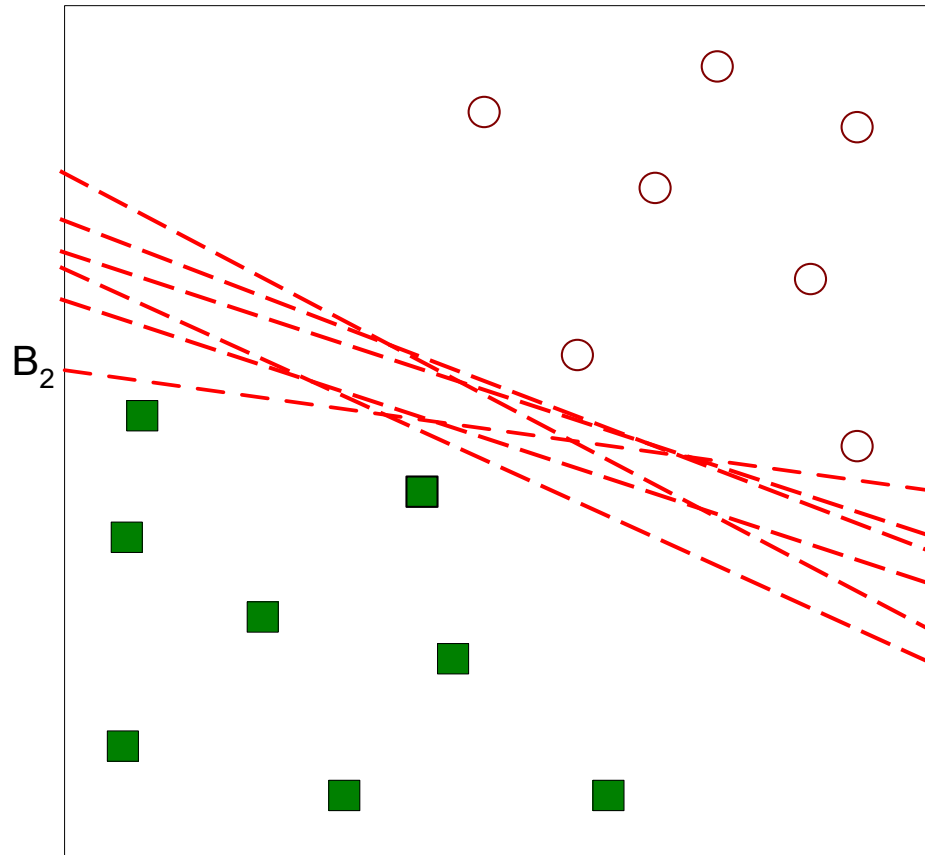
One Possible Solution

Support Vector Machines



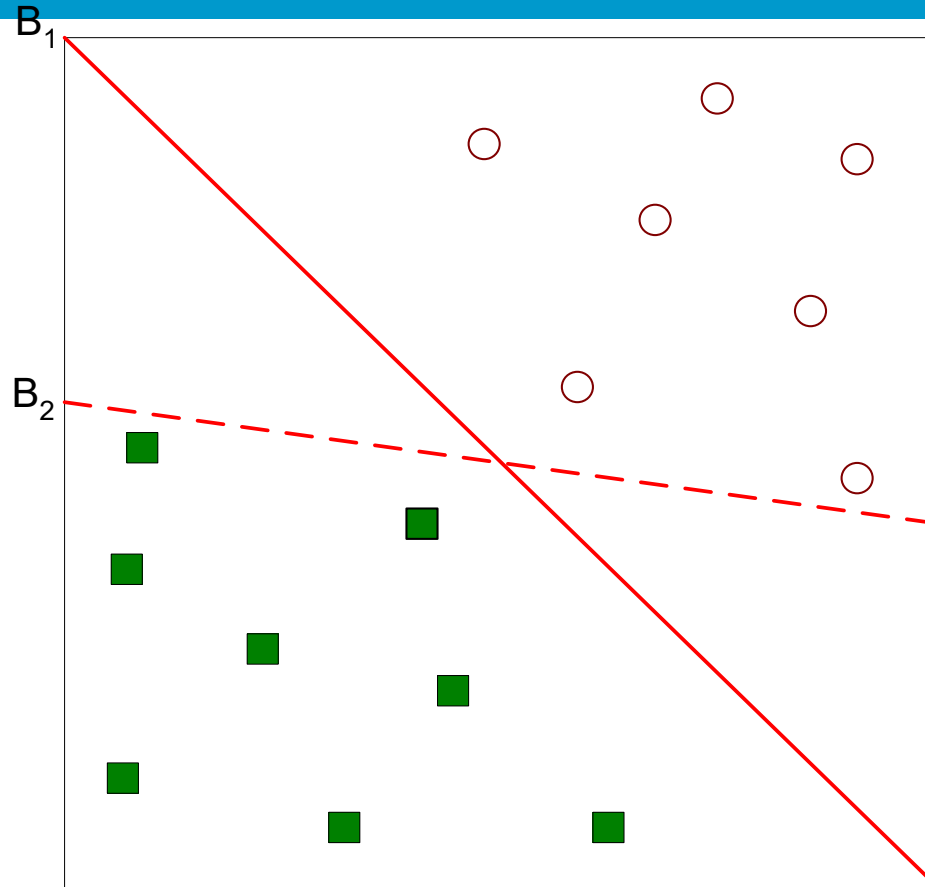
Another possible solution

Support Vector Machines



Other possible solutions

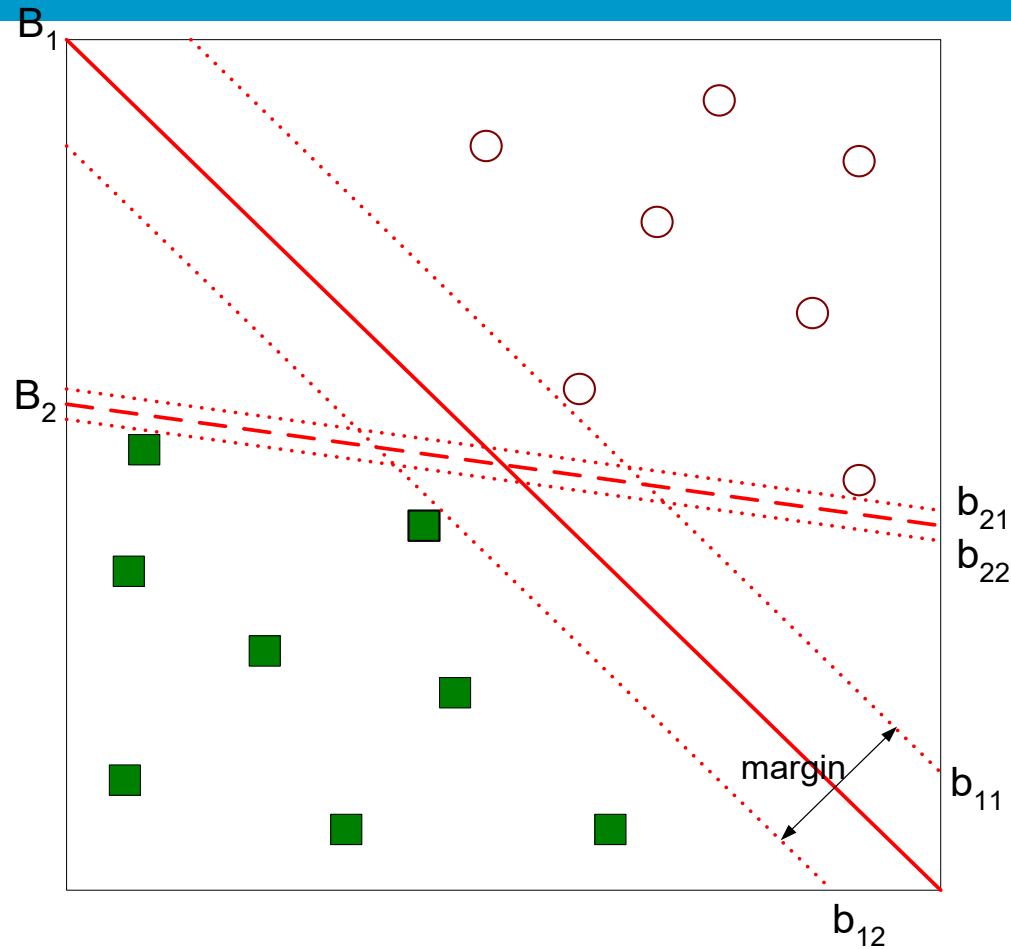
Support Vector Machines



❖ Which one is better? B_1 or B_2 ?

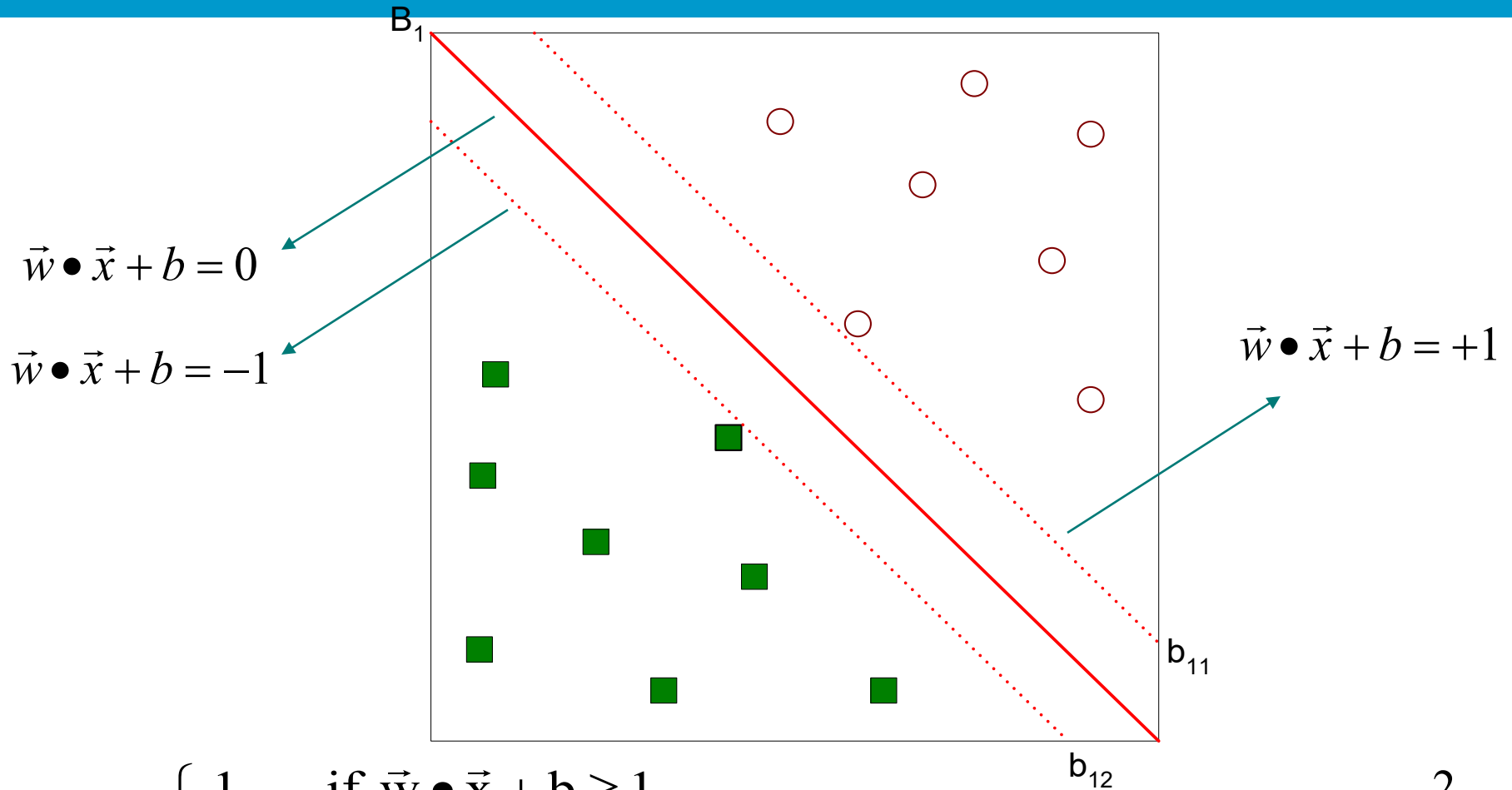
❖ How do you define better?

Support Vector Machines



- ❖ Find hyperplane **maximizes** the margin $\Rightarrow B_1$ is better than B_2

Support Vector Machines



$$f(\vec{x}) = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x} + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x} + b \leq -1 \end{cases}$$

$$\text{Margin} = \frac{2}{\|\vec{w}\|}$$

Learning Linear SVM

- Objective is to maximize: $\text{Margin} = \frac{2}{\|\vec{w}\|}$
 - Which is equivalent to minimizing: $L(\vec{w}) = \frac{\|\vec{w}\|^2}{2}$
 - Subject to the following constraints:

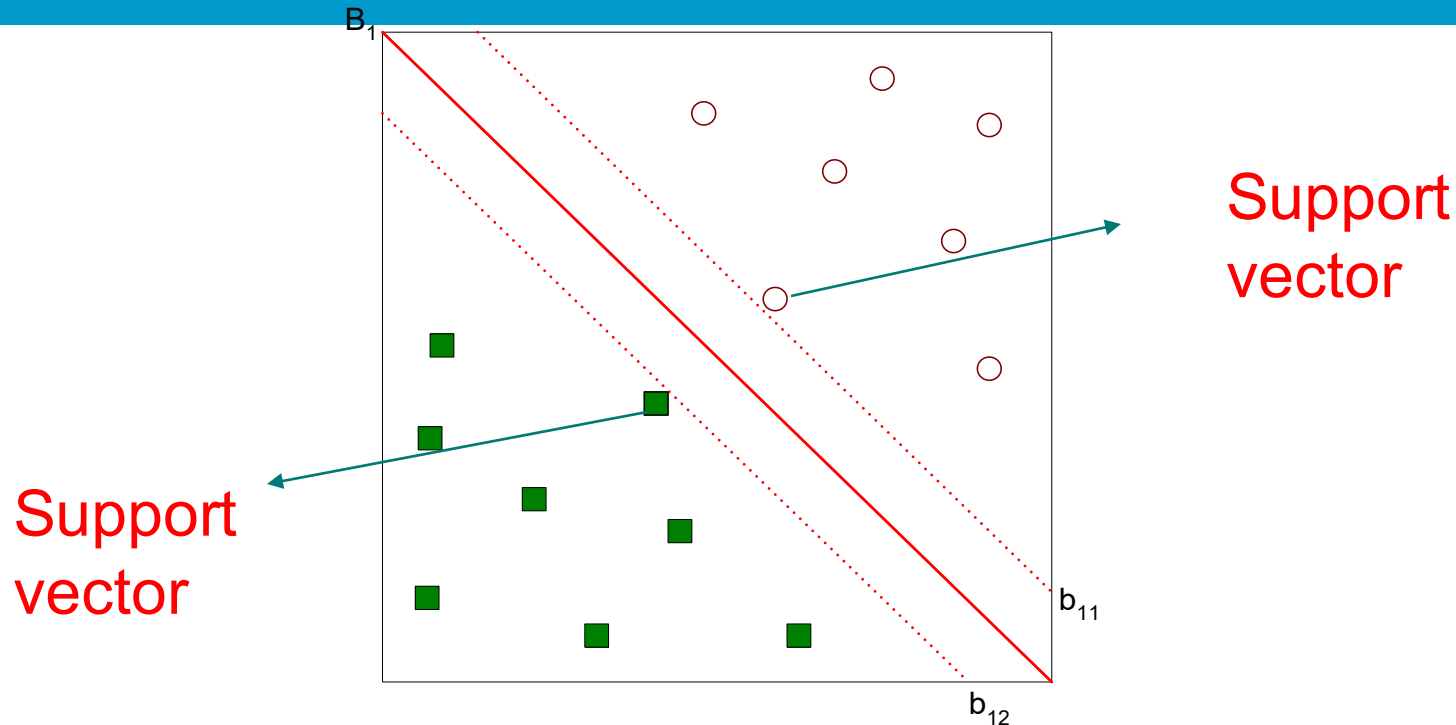
$$y_i = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 \end{cases}$$

or

$$y_i(\mathbf{w} \bullet \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, N$$

- ◆ This is a constrained optimization problem
 - Solve it using Lagrange multiplier method

Support Vector



Support vector: data point located along the margin boundary.

- Support vectors shape the boundaries and decide the SVM model
- More non-support vectors will not change the model parameters (the boundary)

Learning Linear SVM

❖ Decision boundary depends only on support vectors

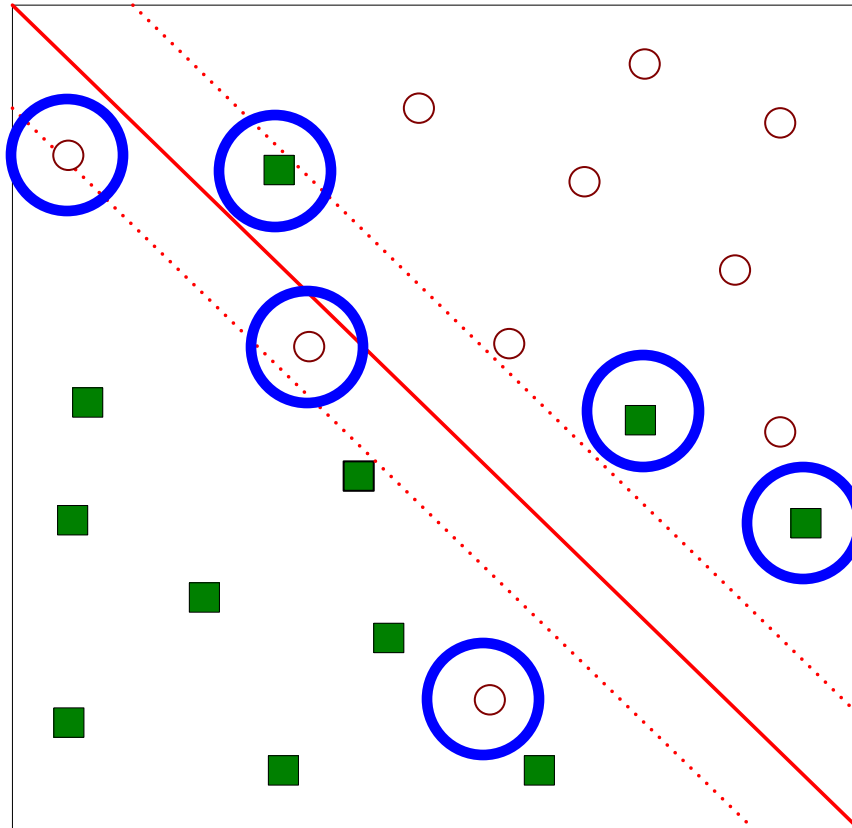
☞ If you have data set with same support vectors, decision boundary will not change

❖ How to classify using SVM once $\mathbf{w}$ and b are found? Given a test record, $\mathbf{x}_i$

$$f(\vec{x}_i) = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 \end{cases}$$

Support Vector Machines

❖ What if the problem is not linearly separable?



Support Vector Machines

- ❖ What if the problem is not linearly separable?

☞ Introduce slack variables

- ❖ Need to minimize:
$$L(w) = \frac{\|\vec{w}\|^2}{2} + C \left(\sum_{i=1}^N \xi_i^k \right)$$

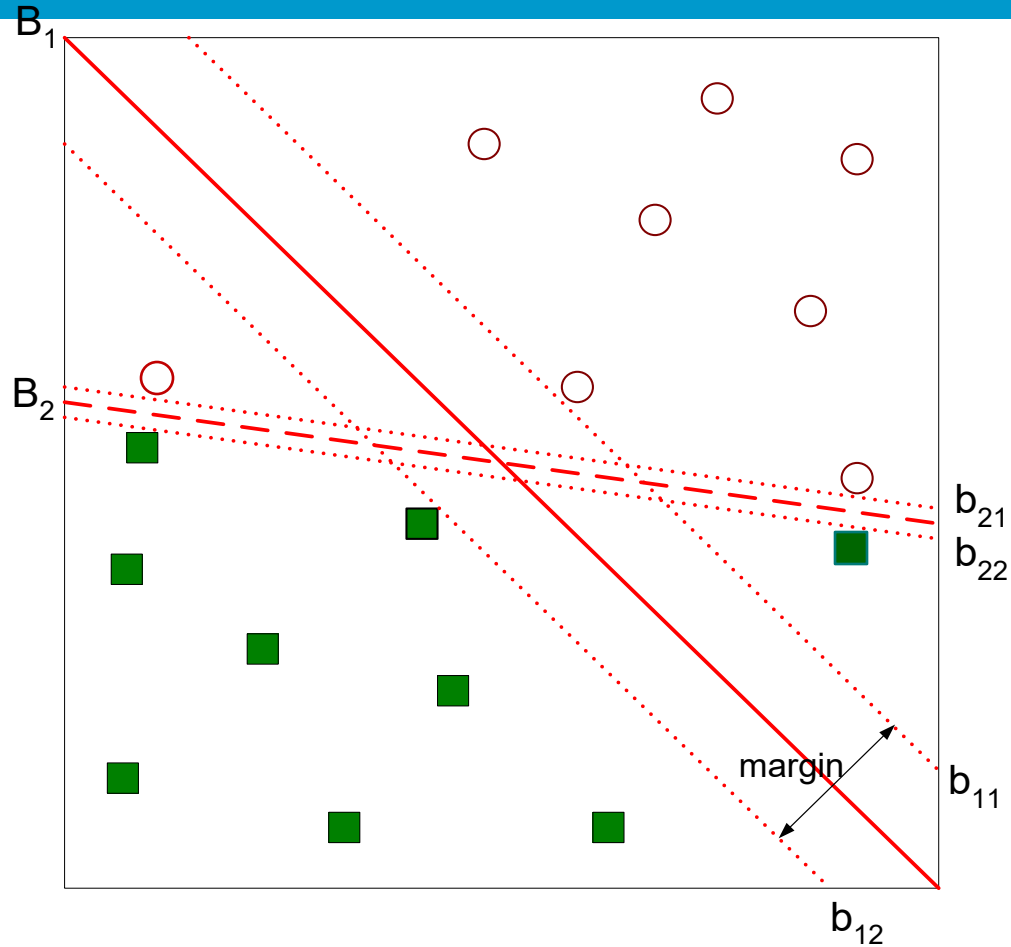
C and k are user-specified parameters representing the penalty of misclassifying the training instances.

- ❖ Subject to:

$$y_i = \begin{cases} 1 & \text{if } \vec{w} \bullet \vec{x}_i + b \geq 1 - \xi_i \\ -1 & \text{if } \vec{w} \bullet \vec{x}_i + b \leq -1 + \xi_i \end{cases}$$

- ❖ If k is 1 or 2, this leads to same objective function as linear SVM but with different constraints (see textbook)

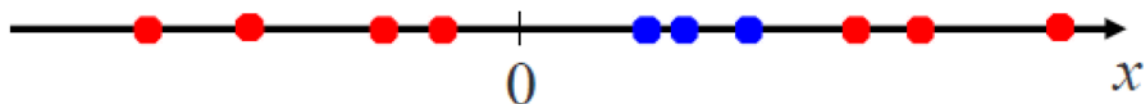
Support Vector Machines



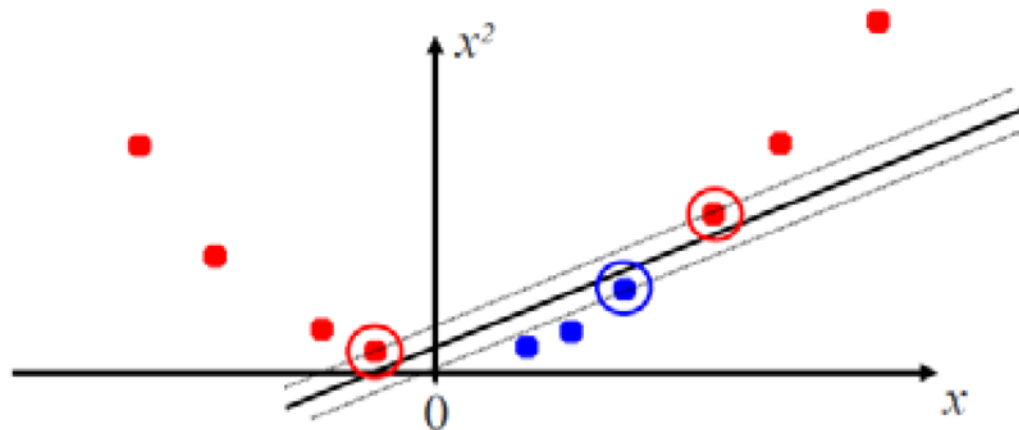
Find the hyperplane that optimizes both factors

Nonlinear SVM

- Is this data linearly-separable?

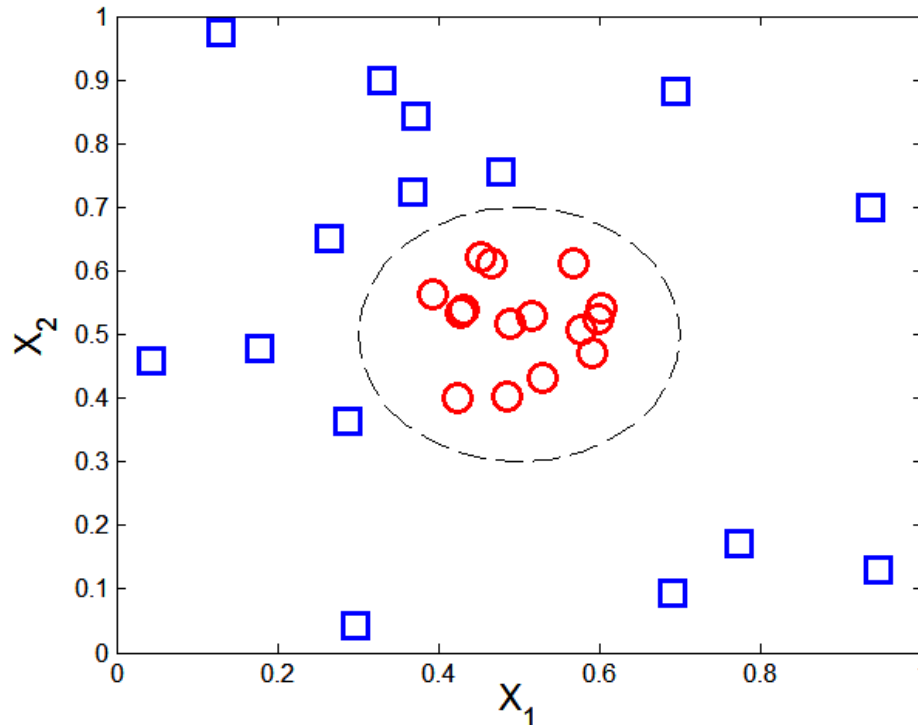


- How about a quadratic mapping $\phi(x_i)$?



Nonlinear Support Vector Machines

❖ What if decision boundary is not linear?



$$y(x_1, x_2) = \begin{cases} 1 & \text{if } \sqrt{(x_1 - 0.5)^2 + (x_2 - 0.5)^2} > 0.2 \\ -1 & \text{otherwise} \end{cases}$$

Nonlinear Support Vector Machines

- ❖ Trick: Transform data into higher dimensional space – Kernel function mapping

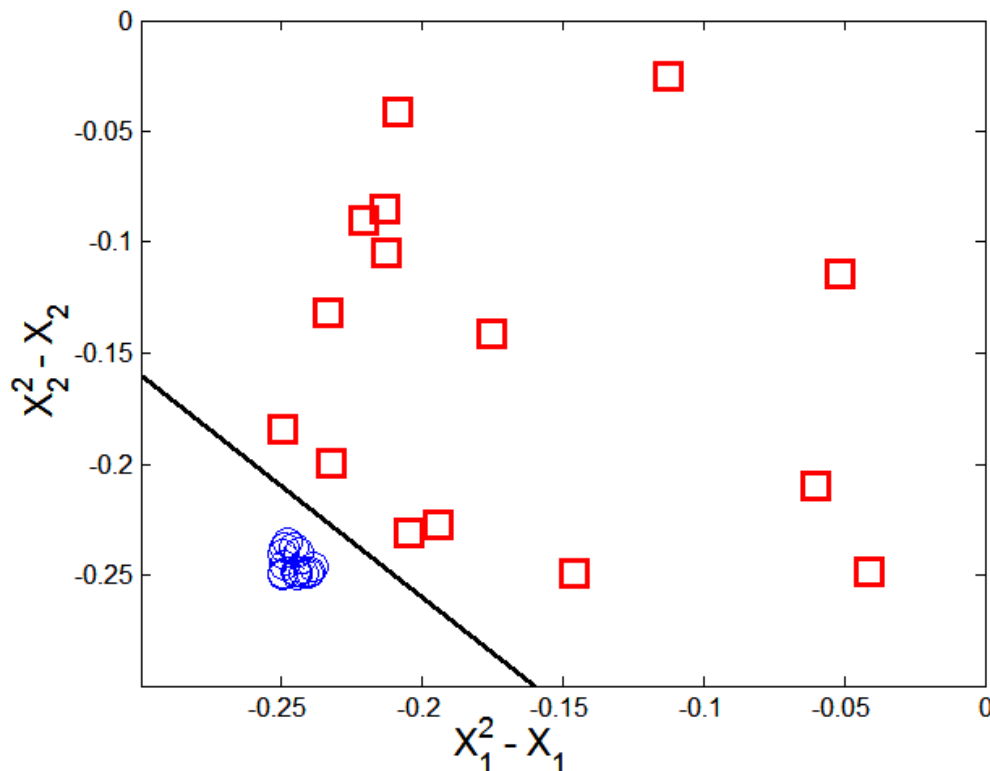
$$x_1^2 - x_1 + x_2^2 - x_2 = -0.46.$$

$$\Phi : (x_1, x_2) \longrightarrow (x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, 1).$$

$$w_4x_1^2 + w_3x_2^2 + w_2\sqrt{2}x_1 + w_1\sqrt{2}x_2 + w_0 = 0.$$

Decision boundary:

$$\vec{w} \bullet \Phi(\vec{x}) + b = 0$$



Learning Nonlinear SVM

❖ Kernel Trick:

$$\Phi(\mathbf{x}_i) \bullet \Phi(\mathbf{x}_j) = K(\mathbf{x}_i, \mathbf{x}_j)$$

$K(\mathbf{x}_i, \mathbf{x}_j)$ is a kernel function (expressed in terms of the coordinates in the original space)

❖ Examples:

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y} + 1)^p$$

$$K(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x} - \mathbf{y}\|^2 / (2\sigma^2)}$$

$$K(\mathbf{x}, \mathbf{y}) = \tanh(k\mathbf{x} \cdot \mathbf{y} - \delta)$$

Characteristics of SVM

- ❖ Since the learning problem is formulated as a convex optimization problem, efficient algorithms are available to find the global minima of the objective function (many of the other methods use greedy approaches and find locally optimal solutions)
- ❖ Overfitting is addressed by maximizing the margin of the decision boundary, but the user still needs to provide the type of kernel function and cost function
- ❖ Difficult to handle missing values
- ❖ Robust to noise
- ❖ High computational complexity for building the model

Brief Summary

- ❖ Comparison between logistic regression and support vector machine
- ❖ How does SVM build a separation boundary

IS6400 Business Data Analytics

Dr. LIU Junming

City University of Hong Kong

Lecture 6

- ☐ Introduction to classification
- ☐ Ensemble Learning
- ☐ Support Vector Machine
- ☒ **Imbalanced classification**

Reading Material:

- ☐ DM Chapter 4,5

Class Imbalance Problem

- ❖ Lots of classification problems where the classes are skewed (more records from one class than another)
 - ❧ Credit card fraud
 - ❧ Intrusion detection
 - ❧ Defective products in manufacturing assembly line

Challenges

- ❖ Evaluation measures such as accuracy is not well-suited for imbalanced class
- ❖ Detecting the rare class is like finding needle in a haystack

Challenges - Example

For 1000 bank transactions, only 1 transaction is a fraud transaction

I have a classifier with a classification accuracy equals to 99.9%, is this a good classifier?

Challenges - Example

For 1000 bank transactions, only 1 transaction is a fraud transaction

I have a classifier with a classification accuracy equals to 99.9%, is this a good classifier?

A stupid classifier who guess all transactions are not fraud has an accuracy of 99.9%!!

Confusion Matrix

❖ Confusion Matrix:

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
Class=Yes	a (TP)	b (FN)
	c (FP)	d (TN)

a: TP (true positive)

b: FN (false negative)

c: FP (false positive)

d: TN (true negative)

Accuracy

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
Class=Yes	a (TP)	b (FN)
	c (FP)	d (TN)

❖ Most widely-used metric:

$$\text{Accuracy} = \frac{a + d}{a + b + c + d} = \frac{TP + TN}{TP + TN + FP + FN}$$

Problem with Accuracy

- ❖ Consider a 2-class problem
 - ❧ Number of Class 0 examples = 9990
 - ❧ Number of Class 1 examples = 10
- ❖ If a model predicts everything to be class 0, accuracy is $9990/10000 = 99.9\%$
 - ❧ This is misleading because the model does not detect any class 1 example
 - ❧ Detecting the rare class is usually more interesting (e.g., frauds, intrusions, defects, etc)

Alternative Measures

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
	Class=Yes a (TP)	b (FN)
Class=No	c (FP)	d (TN)

$$\text{Precision (p)} = \frac{a}{a + c}$$

$$\text{Recall (r)} = \frac{a}{a + b} = TPR$$

$$\text{F - measure (F)} = \frac{2rp}{r + p} = \frac{2a}{2a + b + c}$$

Alternative Measures

ACTUAL CLASS	PREDICTED CLASS	
	Class=Yes	Class=No
	Class=Yes a=10 (TP)	b=20 (FN)
Class=No	c=50 (FP)	d=10000 (TN)

$$\text{Precision (p)} = \frac{a}{a + c}$$

$$\text{Recall (r)} = \frac{a}{a + b} = \text{TPR}$$

$$\text{F - measure (F)} = \frac{2rp}{r + p} = \frac{2a}{2a + b + c}$$

If you are a bank manager,
Which number do you care most?
a, b, c, d?

Cost Matrix

	PREDICTED CLASS		
ACTUAL CLASS		Class=Yes	Class=No
	Class=Yes	f(Yes, Yes)	f(Yes, No)
	Class=No	f(No, Yes)	f(No, No)

$C(i,j)$: Cost of misclassifying class i example as class j

Cost Matrix	PREDICTED CLASS		
ACTUAL CLASS	$C(i, j)$	Class=Yes	Class=No
	Class=Yes	$C(\text{Yes, Yes})$	$C(\text{Yes, No})$
	Class=No	$C(\text{No, Yes})$	$C(\text{No, No})$

$$\text{Cost} = \sum C(i, j) \times f(i, j)$$

Computing Cost of Classification

Cost Matrix	PREDICTED CLASS		
ACTUAL CLASS	C(i,j)	+	-
	+	-1	100
	-	1	0

Model M_1	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	150	40
	-	60	250

Accuracy = 80%

Cost = 3910

Model M_2	PREDICTED CLASS		
ACTUAL CLASS		+	-
	+	250	45
	-	5	200

Accuracy = 90%

Cost = 4255

Handling Class Imbalanced Problem

- ❖ Class-based ordering (e.g. RIPPER)
 - ⌘ Rules for rare class have higher priority
- ❖ Cost-sensitive classification
 - ⌘ Misclassifying rare class as majority class is more expensive than misclassifying majority as rare class
- ❖ Sampling-based approaches
 - ⌘ Only rebalancing the TRAINING DATA!

Class-based Ordering

❖ RIPPER:

- ❧ Generate a set of 'if-then' rules to predict the target category.

❖ An example of detecting Spam:

- ❧ Rule generated: The email contains the word 'free' or 'win'. → Spam

- ❧ Prioritized rule generation for a small number of classes (spam), suitable for processing unbalanced data

Cost Sensitive Classification

❖ General decision rule:

❧ Classify test record x as class k if

$$k = \arg \min_j \sum_i p(i | x) \times C(i, j)$$

❖ 2-class:

❧ $\text{Cost}(+) = p(+|x) C(+,+) + p(-|x) C(-,+)$

❧ $\text{Cost}(-) = p(+|x) C(+,-) + p(-|x) C(-,-)$

❧ Decision rule: classify x as $+$ if $\text{Cost}(+) < \text{Cost}(-)$

Sampling-based Approaches

- ❖ Modify the distribution of training data so that rare class is well-represented in training set
 - ⌘ Undersample the majority class
 - ⌘ Oversample the rare class

ONLY for training. Data distribution of the testing set should be the same as the population

Sampling-based Approaches

❖ Under-sampling

- ❧ Prototype generation: generate a smaller new set (not selected from the original set)
- ❧ Prototype selection: select samples from the original set
- ❧ Instance Hardness Threshold: measure how difficult it is to classify an instance correctly, remove these hard instances from the dataset.

Sampling-based Approaches

❖ Over-sampling

- ❧ Naïve random over-sampling: generate new samples by randomly sampling with replacement the current available ones
- ❧ SMOTE and ADASYN: duplicating some of the original samples of the minority class
 - ❖ ADASYN: focuses more on samples next to wrongly classified samples next to the original ones
 - ❖ SMOTE: not make distinction between easy and hard samples to be classified

Sampling-based Approaches

- ❖ Combination of Over- and Under-sampling
 - ❧ Over-sampling may bring noisy samples when interpolating new points
 - ❧ Add cleaning methods to clean the space resulting from over-sampling
 - ❧ E.g. SMOTETomek and SMOTEENN